

**Analysis of Skill Gap and Labour Market
Demand using NLP**

A

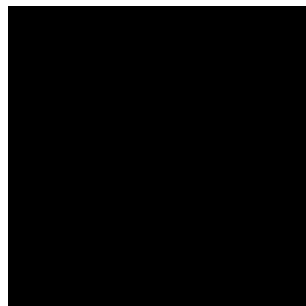
Project-I Report

Submitted

In Partial Fulfilment

For the award of the Degree of

**Bachelor of Technology
(VII Semester)**



Supervised By:

Guide Name

Designation

Submitted By:

Student Name

Roll No:

**Department of Computer Science and
Engineering**

**Jodhpur Institute of Engineering and Technology
(Autonomous)**

JIET Universe, Jodhpur

Session 2026-27

Certificate

This is to certify that the project entitled “**Analysis of Skill Gap and Labour Market Demand using NLP**” has been carried out by the students of **Jodhpur Institute of Engineering & Technology, Jodhpur** under my guidance and supervision in partial fulfillment of the degree of Bachelor of Technology in Computer Science and Engineering of Bikaner Technical University during the academic year of 2026-27.

Date:

Supervisor's Name

Place:

Designation

Acknowledgment

I want to express my sincere gratitude to all those who have been instrumental in the completion of my project report on “Analysis of Skill Gap and Labour Market Demand using NLP”. Working on this project has been a huge learning experience, and I am grateful to everyone who helped us along the way.

An acknowledgment is extended to my guide for their remarkable contributions. Their expertise, feedback, and dedication played a crucial role in achieving our goals. Their guidance kept us on track whenever we faced bugs or logic issues, especially during the backend and ML integration phases.

Furthermore, I am grateful to my academic institution, Jodhpur Institute of Engineering and Technology, for providing the opportunity and the lab facilities to work on this real-world problem statement. The environment provided by the institution played a big role in the successful completion of this project.

Lastly, I express my gratitude to my teammates and friends whose direct and indirect support contributed to finishing this project on time.

Student Name

Roll No:

Date :

Contents

1	Introduction	7
1.1	Background of the Study	7
1.2	Problem Statement	8
1.3	Objectives of the Project	8
1.4	Scope of the Project	9
1.5	Significance of the Study	9
1.6	Methodology Overview	9
2	Literature Survey	11
2.1	Introduction	11
2.2	Labour Market Intelligence Systems	11
2.3	Text Mining in Job Descriptions	12
2.4	Traditional Curriculum Updates	12
2.5	Machine Learning in Education	13
2.6	Research Gap Identified	13
3	Requirement Specification & Methodology	14
3.1	Software Development Life Cycle (SDLC) Model	14
3.2	Use Case Diagrams	14
3.3	Interfaces	15

3.3.1	Software Interfaces	15
3.3.2	Hardware Interfaces	15
3.3.3	Communication Interfaces	15
3.4	Methodology Used	16
4	Work Distribution	17
4.1	Task Allocation	17
4.2	Module Details	17
5	Design Document	19
5.1	Functional Description	19
5.2	Data Description	19
5.3	User Interface Design	20
6	Experimental Setup	21
6.1	Development Environment	21
6.2	Execution Process	21
7	Test Plan Document	22
7.1	Test Strategy	22
7.2	Test Cases and Status Report	22
8	Results	23
8.1	Outputs and Observations	23
8.2	Performance Analysis	23
9	Conclusion & Future Work	24
9.1	Conclusion	24

9.2 Limitations	24
9.3 Future Work	24
10 Glossary	26
11 References	27
Appendix A: Source Code	28

Chapter 1

Introduction

1.1 Background of the Study

In recent years, the gap between what colleges teach and what the industry actually needs has become a major issue. Many students graduate with degrees but struggle to find jobs because they lack the specific technical skills that companies are actively hiring for. At the same time, companies complain that they cannot find industry-ready freshers.

To solve this, we need a way to track the exact skills demanded by the market and compare them directly against college syllabuses. By automating this process using Natural Language Processing (NLP), we can extract skills from job descriptions and college PDFs, and give students and colleges a clear, mathematical gap report. This project, SkillBridge AI, is built to bridge this exact gap using a modern tech stack (React, Node.js, Python Flask).

1.2 Problem Statement

Colleges currently design their curriculum based on outdated academic guidelines, while the IT industry evolves every few months. There is no automated feedback loop between employers and educational institutes. Traditional methods of updating a syllabus rely on slow, manual surveys that take years to implement. Therefore, the problem is to build an automated Labour Market Intelligence platform that can read live job postings, extract required skills, analyze college syllabuses, and output actionable training plans and gap reports in real-time.

1.3 Objectives of the Project

The primary objective is to design and develop a web-based platform for skill gap analysis. The sub-objectives include:

- Collecting live job postings and extracting skills using NLP and regular expressions.
- Parsing college syllabus text to identify currently taught subjects.
- Calculating an Industry Readiness Score for students based on their personal skills.
- Generating district-level training plans for colleges to upgrade their labs and faculty.

- Providing a dashboard for companies to submit future skill requirements.

1.4 Scope of the Project

The scope of this project is limited to the IT and Software Engineering sectors for the prototype phase. It handles text-based syllabus inputs and compares them against a predefined taxonomy of over 200 IT skills (like Python, React, Docker, AWS). The system focuses on providing three distinct dashboards: one for Students, one for Colleges, and one for Companies, managed by an Admin portal.

1.5 Significance of the Study

This project provides a practical tool for educational boards and universities to modernize their courses. Instead of guessing what to teach, colleges can look at the dashboard and see exactly what skills are rising (e.g., Generative AI) and what is declining. For students, it provides a reality check on their employability and suggests direct courses to fix their skill gaps.

1.6 Methodology Overview

The methodology followed in this project includes:

1. Requirement gathering from the SIH problem statement.
2. Designing the database schema using Prisma and SQLite.

3. Building the ML engine in Python (Flask) for skill extraction using a taxonomy dictionary.
4. Developing the backend API using Node.js and Express.
5. Creating the frontend user interface using React and modern CSS.
6. Integrating all tiers and deploying for testing.

Chapter 2

Literature Survey

2.1 Introduction

This chapter presents a review of existing methods used for skill extraction and curriculum analysis. We looked at how different researchers have attempted to solve the academic-industry gap using technology.

2.2 Labour Market Intelligence Systems

Labour Market Intelligence (LMI) refers to systems that analyze job market data to find trends. Most existing LMI systems are heavily enterprise-focused and expensive. They scrape LinkedIn or Indeed to sell data to recruiters. However, there are very few systems designed specifically to give this data back to colleges in the form of a "Syllabus Gap Report."

2.3 Text Mining in Job Descriptions

Extracting skills from job descriptions is a classic text mining problem. Job descriptions are usually unstructured text.

- **Keyword Matching:** The simplest approach is to keep a list of known skills and search for them in the text. This is fast but misses variations.
- **Named Entity Recognition (NER):** Advanced models use NLP (like Spacy or custom BERT models) to identify skills contextually.

For our project, we decided to use a highly optimized dictionary-based Regex approach (Keyword Matching with aliases) because it is lightweight, fast, and doesn't require a GPU to run on a college server.

2.4 Traditional Curriculum Updates

Traditionally, a university forms a committee every 3 to 4 years to update the syllabus. They might invite a few industry experts for a meeting. This process is manual, biased by the opinions of a few people, and extremely slow. By the time a new syllabus is approved, the technology might already be obsolete.

2.5 Machine Learning in Education

Machine learning is being increasingly used in EdTech for personalized learning. Systems can recommend courses based on a student's past performance. In our system, we use a recommendation engine to suggest specific courses (e.g., from Coursera or NPTEL) to bridge the exact skills a student is missing based on the job they want.

2.6 Research Gap Identified

Based on the literature survey, the main gaps are:

- Most NLP models for skill extraction are too heavy to deploy on cheap college servers.
- Existing LMI tools don't directly compare job skills against a college syllabus PDF.
- There is no unified platform where students, colleges, and companies can interact directly regarding skill demands.

This project aims to fill this gap by providing a lightweight, full-stack solution.

Chapter 3

Requirement Specification & Methodology

3.1 Software Development Life Cycle (SDLC) Model

We chose the Agile methodology for this project. Since we had to build a complex 3-tier architecture (React, Node, Python) in a short time for the hackathon, Agile allowed us to develop features in sprints. We started with the basic login and database, then moved to the Python ML engine, and finally built the React dashboards.

3.2 Use Case Diagrams

The system has four main actors: Admin, Student, College, and Company.

- **Student:** Can view market trends, do personal skill assessments, and check career pathways.
- **College:** Can upload syllabus, view gap reports, generate train-

ing plans, and submit placement data.

- **Company:** Can post jobs, submit industry consultations, and validate reports.
- **Admin:** Views overall statistics and manages demo data seeding.

3.3 Interfaces

3.3.1 Software Interfaces

- **Frontend:** React.js, Vite, Axios, React Router.
- **Backend:** Node.js, Express.js, Prisma ORM.
- **ML Engine:** Python, Flask, Regular Expressions (re module).
- **Database:** SQLite (for local development and easy setup).

3.3.2 Hardware Interfaces

The project does not require specific hardware sensors. It runs on any standard computer.

- Minimum 4GB RAM, Core i3 processor.
- Internet connection for fetching course recommendations.

3.3.3 Communication Interfaces

The frontend communicates with the Node.js backend via HTTP REST APIs (port 5000). The Node.js backend communicates with

the Python ML Engine via HTTP POST/GET requests (port 5001). Data is exchanged in JSON format.

3.4 Methodology Used

1. **Data Collection:** We created a taxonomy of 200+ IT skills categorized into Web Dev, AI/ML, Cloud, etc.
2. **Preprocessing:** User inputs (syllabus text or job text) are converted to lowercase and cleaned of special characters. We apply an alias map (e.g., replacing 'mysql' with 'sql') to standardize terms.
3. **Feature Extraction:** The Python script uses word boundary regex (`\b`) to match taxonomy skills inside the text.
4. **Analysis:** The extracted syllabus skills are mathematically subtracted from the extracted job skills to find the missing gaps.

Chapter 4

Work Distribution

4.1 Task Allocation

The work was divided to ensure parallel development of the frontend and backend.

Module	Description	Status
Database Design	Creating Prisma schema models	Completed
Node Backend	User auth, job posting routes, analytics	Completed
Python ML Engine	Skill extraction, alias matching, gap logic	Completed
React Frontend	Dashboards for all 4 user roles	Completed
UI/UX Design	CSS styling, AI-themed neon aesthetic	Completed
Integration	Connecting React to Node and Node to Python	Completed

Table 4.1: Task Allocation

4.2 Module Details

- **Auth Module:** Handles JWT-based login and registration.
- **Dashboard Module:** Renders different UI components based on the user's role.

- **Analysis Module (Python):** Contains the `SKILL_TAXONOMY`, alias mapping, and obsolete skill detection.

Chapter 5

Design Document

5.1 Functional Description

The system follows a microservice-like architecture. The React app is the client. The Node.js server acts as the main API gateway and handles database operations. For heavy text analysis, Node.js forwards the text to the Python Flask server, which returns the extracted JSON data back to Node.js.

5.2 Data Description

We used Prisma ORM with SQLite. The main tables are:

- **User:** Stores email, hashed password, role, and location.
- **JobPosting:** Stores job title, description, sector, and salary.
- **GapReport:** Stores the analysis results (missing skills, alignment score).
- **TrainingPlan:** Stores generated training plans for districts.

5.3 User Interface Design

The user interface is designed with a modern, dark-mode, AI-centric aesthetic. It uses neon cyan and purple highlights, glassmorphism panels, and AI-themed icons (like neural networks and data matrices) to look futuristic.

Chapter 6

Experimental Setup

6.1 Development Environment

- **OS:** Windows 11
- **Node version:** v18+
- **Python version:** 3.10+
- **IDE:** Visual Studio Code

6.2 Execution Process

To run the system locally, three terminal windows are required:

1. **Frontend:** `cd frontend → npm run dev` (runs on port 5173)
2. **Backend:** `cd backend → npx tsx src/index.ts` (runs on port 5000)
3. **ML Engine:** `cd ml-engine → python app.py` (runs on port 5001)

Chapter 7

Test Plan Document

7.1 Test Strategy

We performed manual unit testing on the API endpoints using tools like Postman, and end-to-end UI testing through the browser.

7.2 Test Cases and Status Report

S.No.	Module	Status	Remarks
1	User Login & Auth	Pass	JWT tokens working correctly.
2	Job Posting Creation	Pass	Data saves to SQLite successfully.
3	NLP Skill Extraction	Pass	Aliases (e.g. mysql $\rightarrow$ sql) matched correctly.
4	Personal Assessment	Pass	Calculates correct alignment score.
5	AI UI Theme applied	Pass	Neon colors and icons display correctly.

Table 7.1: Test Case Status Report

Chapter 8

Results

8.1 Outputs and Observations

The system successfully identifies missing skills. For example, when a job description required "MySQL, Python, Power BI" and the syllabus only contained "Python", the system correctly flagged MySQL (normalized as SQL) and Power BI as missing skills. It also successfully generated a Training Plan suggesting that the college needs to acquire "Data Science Trainers" and "Cloud Sandbox Accounts".

8.2 Performance Analysis

The regex-based skill extraction in Python takes less than 50 milliseconds per request, making it extremely fast and suitable for real-time gap analysis without the need for expensive GPU servers.

Chapter 9

Conclusion & Future Work

9.1 Conclusion

This project successfully designed and implemented a Labour Market Intelligence platform. The objectives outlined in Chapter 1 were achieved. We proved that by using a well-structured skill taxonomy and alias mapping, we can accurately compare student knowledge against industry requirements and generate actionable gap reports.

9.2 Limitations

- The current taxonomy is hardcoded to 200 IT skills.
- It currently does not parse actual PDF files directly in the UI (users have to copy-paste syllabus text).

9.3 Future Work

- Implementing PDF and OCR parsing so colleges can directly upload their syllabus documents.

- Moving the skill taxonomy to a dynamic cloud database so it can be updated by admins without changing code.
- Integrating a real LLM (like Gemini or OpenAI) to handle complex, unstructured job descriptions instead of relying solely on regex.

Chapter 10

Glossary

Term	Full Form	Description
AI	Artificial Intelligence	Logic that simulates human decision making.
NLP	Natural Language Processing	Analyzing text data to extract meaning.
LMI	Labour Market Intelligence	Data regarding job market demand.
UI	User Interface	The visual layout of the application.

Table 10.1: Glossary of Terms

Chapter 11

References

1. React Documentation, [Online]. Available: <https://react.dev/>
2. Flask Documentation, [Online]. Available: <https://flask.palletsprojects.com/>
3. Prisma ORM, "Next-generation Node.js and TypeScript ORM", [Online]. Available: <https://www.prisma.io/>
4. Previous Year SIH Problem Statements regarding Skill Gap Analysis.

Appendix A: Source Code

Source Code: app.py

```
from flask import Flask, request, jsonify
from flask_cors import CORS

import re

import json

app = Flask(__name__)
CORS(app)

#    MASSIVE SKILL TAXONOMY (200+ skills)
SKILL_TAXONOMY = {

    #    Programming Languages

    "python": {"category": "Programming", "sector": "IT", "demand": "Hi"},
    "java": {"category": "Programming", "sector": "IT", "demand": "Hi"},
    "javascript": {"category": "Programming", "sector": "IT", "demand": "Hi"},
    "typescript": {"category": "Programming", "sector": "IT", "demand": "Hi"},
    "c++": {"category": "Programming", "sector": "IT", "demand": "Med"},
    "c#": {"category": "Programming", "sector": "IT", "demand": "Medi"}
```

```

"c programming": {"category": "Programming", "sector": "IT", "demand": "High"},
"go": {"category": "Programming", "sector": "IT", "demand": "High"},
"golang": {"category": "Programming", "sector": "IT", "demand": "High"},
"rust": {"category": "Programming", "sector": "IT", "demand": "Medium"},
"kotlin": {"category": "Programming", "sector": "IT", "demand": "Medium"},
"swift": {"category": "Programming", "sector": "IT", "demand": "Medium"},
"r programming": {"category": "Programming", "sector": "IT", "demand": "Medium"},
"php": {"category": "Programming", "sector": "IT", "demand": "Low"},
"ruby": {"category": "Programming", "sector": "IT", "demand": "Low"},
"scala": {"category": "Programming", "sector": "IT", "demand": "Low"}

```

Web Development

```

"html": {"category": "Web Development", "sector": "IT", "demand": "High"},
"css": {"category": "Web Development", "sector": "IT", "demand": "High"},
"react": {"category": "Web Development", "sector": "IT", "demand": "High"},
"react.js": {"category": "Web Development", "sector": "IT", "demand": "High"},
"angular": {"category": "Web Development", "sector": "IT", "demand": "High"},
"vue.js": {"category": "Web Development", "sector": "IT", "demand": "High"},
"vue": {"category": "Web Development", "sector": "IT", "demand": "High"},
"next.js": {"category": "Web Development", "sector": "IT", "demand": "High"},
"node.js": {"category": "Web Development", "sector": "IT", "demand": "High"},
"node": {"category": "Web Development", "sector": "IT", "demand": "High"},
"express": {"category": "Web Development", "sector": "IT", "demand": "High"},
"express.js": {"category": "Web Development", "sector": "IT", "demand": "High"},
"django": {"category": "Web Development", "sector": "IT", "demand": "High"}

```

```

"flask": {"category": "Web Development", "sector": "IT", "demand": "High"},
"spring boot": {"category": "Web Development", "sector": "IT", "demand": "High"},
"asp.net": {"category": "Web Development", "sector": "IT", "demand": "High"},
"tailwind css": {"category": "Web Development", "sector": "IT", "demand": "High"},
"bootstrap": {"category": "Web Development", "sector": "IT", "demand": "High"},
"graphql": {"category": "Web Development", "sector": "IT", "demand": "High"},
"rest api": {"category": "Web Development", "sector": "IT", "demand": "High"},
"websocket": {"category": "Web Development", "sector": "IT", "demand": "High"}

```

Databases

```

"sql": {"category": "Database", "sector": "IT", "demand": "High"},
"mysql": {"category": "Database", "sector": "IT", "demand": "High"},
"postgresql": {"category": "Database", "sector": "IT", "demand": "High"},
"mongodb": {"category": "Database", "sector": "IT", "demand": "High"},
"redis": {"category": "Database", "sector": "IT", "demand": "High"},
"elasticsearch": {"category": "Database", "sector": "IT", "demand": "High"},
"firebase": {"category": "Database", "sector": "IT", "demand": "Medium"},
"dynamodb": {"category": "Database", "sector": "IT", "demand": "Medium"},
"cassandra": {"category": "Database", "sector": "IT", "demand": "Medium"},
"oracle": {"category": "Database", "sector": "IT", "demand": "Medium"},
"sqlite": {"category": "Database", "sector": "IT", "demand": "Low"}

```

Cloud & DevOps

```

"aws": {"category": "Cloud", "sector": "IT", "demand": "High"},
"azure": {"category": "Cloud", "sector": "IT", "demand": "High"},

```

```

"google cloud": {"category": "Cloud", "sector": "IT", "demand": "High"},
"gcp": {"category": "Cloud", "sector": "IT", "demand": "High", "trend": "Up"},
"docker": {"category": "DevOps", "sector": "IT", "demand": "High", "trend": "Up"},
"kubernetes": {"category": "DevOps", "sector": "IT", "demand": "High", "trend": "Up"},
"terraform": {"category": "DevOps", "sector": "IT", "demand": "High", "trend": "Up"},
"ansible": {"category": "DevOps", "sector": "IT", "demand": "Medium", "trend": "Stable"},
"jenkins": {"category": "DevOps", "sector": "IT", "demand": "Medium", "trend": "Stable"},
"ci/cd": {"category": "DevOps", "sector": "IT", "demand": "High", "trend": "Up"},
"github actions": {"category": "DevOps", "sector": "IT", "demand": "High", "trend": "Up"},
"linux": {"category": "DevOps", "sector": "IT", "demand": "High", "trend": "Stable"},
"nginx": {"category": "DevOps", "sector": "IT", "demand": "Medium", "trend": "Stable"},
"microservices": {"category": "DevOps", "sector": "IT", "demand": "High", "trend": "Up"},

# AI / ML / Data Science
"machine learning": {"category": "AI/ML", "sector": "IT", "demand": "High", "trend": "Up"},
"deep learning": {"category": "AI/ML", "sector": "IT", "demand": "High", "trend": "Up"},
"artificial intelligence": {"category": "AI/ML", "sector": "IT", "demand": "High", "trend": "Up"},
"nlp": {"category": "AI/ML", "sector": "IT", "demand": "High", "trend": "Up"},
"natural language processing": {"category": "AI/ML", "sector": "IT", "demand": "High", "trend": "Up"},
"computer vision": {"category": "AI/ML", "sector": "IT", "demand": "High", "trend": "Up"},
"tensorflow": {"category": "AI/ML", "sector": "IT", "demand": "High", "trend": "Up"},
"pytorch": {"category": "AI/ML", "sector": "IT", "demand": "High", "trend": "Up"},
"pandas": {"category": "Data Science", "sector": "IT", "demand": "Medium", "trend": "Stable"},
"numpy": {"category": "Data Science", "sector": "IT", "demand": "Medium", "trend": "Stable"},
"scikit-learn": {"category": "Data Science", "sector": "IT", "demand": "Medium", "trend": "Stable"}

```

```
"data analysis": {"category": "Data Science", "sector": "IT", "demand": "High"},
"data visualization": {"category": "Data Science", "sector": "IT", "demand": "High"},
"big data": {"category": "Data Science", "sector": "IT", "demand": "High"},
"hadoop": {"category": "Data Science", "sector": "IT", "demand": "High"},
"spark": {"category": "Data Science", "sector": "IT", "demand": "High"},
"power bi": {"category": "Data Science", "sector": "IT", "demand": "High"},
"tableau": {"category": "Data Science", "sector": "IT", "demand": "High"},
"generative ai": {"category": "AI/ML", "sector": "IT", "demand": "High"},
"llm": {"category": "AI/ML", "sector": "IT", "demand": "High"},
"prompt engineering": {"category": "AI/ML", "sector": "IT", "demand": "High"},
"langchain": {"category": "AI/ML", "sector": "IT", "demand": "Medium"}
```

Cybersecurity

```
"cybersecurity": {"category": "Security", "sector": "IT", "demand": "High"},
"ethical hacking": {"category": "Security", "sector": "IT", "demand": "High"},
"penetration testing": {"category": "Security", "sector": "IT", "demand": "High"},
"network security": {"category": "Security", "sector": "IT", "demand": "High"},
"cryptography": {"category": "Security", "sector": "IT", "demand": "High"},
"soc": {"category": "Security", "sector": "IT", "demand": "Medium"},
"owasp": {"category": "Security", "sector": "IT", "demand": "Medium"}
```

Mobile Development

```
"android": {"category": "Mobile", "sector": "IT", "demand": "High"},
"ios": {"category": "Mobile", "sector": "IT", "demand": "Medium"},
"react native": {"category": "Mobile", "sector": "IT", "demand": "Medium"}
```

```
"flutter": {"category": "Mobile", "sector": "IT", "demand": "High"}
```

Blockchain & Emerging

```
"blockchain": {"category": "Emerging Tech", "sector": "IT", "demand": "High"}
```

```
"web3": {"category": "Emerging Tech", "sector": "IT", "demand": "High"}
```

```
"iot": {"category": "Emerging Tech", "sector": "IT", "demand": "Medium"}
```

```
"internet of things": {"category": "Emerging Tech", "sector": "IT", "demand": "Medium"}
```

```
"ar/vr": {"category": "Emerging Tech", "sector": "IT", "demand": "Medium"}
```

```
"robotics": {"category": "Emerging Tech", "sector": "IT", "demand": "Medium"}
```

```
"quantum computing": {"category": "Emerging Tech", "sector": "IT", "demand": "Medium"}
```

```
"edge computing": {"category": "Emerging Tech", "sector": "IT", "demand": "Medium"}
```

```
"5g": {"category": "Emerging Tech", "sector": "IT", "demand": "Medium"}
```

Software Engineering

```
"git": {"category": "Tools", "sector": "IT", "demand": "High", "trend": "Up"}
```

```
"github": {"category": "Tools", "sector": "IT", "demand": "High", "trend": "Up"}
```

```
"agile": {"category": "Methodology", "sector": "IT", "demand": "High", "trend": "Up"}
```

```
"scrum": {"category": "Methodology", "sector": "IT", "demand": "High", "trend": "Up"}
```

```
"jira": {"category": "Tools", "sector": "IT", "demand": "Medium", "trend": "Stable"}
```

```
"devops": {"category": "DevOps", "sector": "IT", "demand": "High", "trend": "Up"}
```

```
"system design": {"category": "Architecture", "sector": "IT", "demand": "Medium", "trend": "Stable"}
```

```
"design patterns": {"category": "Architecture", "sector": "IT", "demand": "Medium", "trend": "Stable"}
```

```
"data structures": {"category": "CS Fundamentals", "sector": "IT", "demand": "Medium", "trend": "Stable"}
```

```
"algorithms": {"category": "CS Fundamentals", "sector": "IT", "demand": "Medium", "trend": "Stable"}
```

```
"object oriented programming": {"category": "CS Fundamentals", "sector": "IT", "demand": "Medium", "trend": "Stable"}
```

```

"oop": {"category": "CS Fundamentals", "sector": "IT", "demand":
"operating systems": {"category": "CS Fundamentals", "sector": "I
"computer networks": {"category": "CS Fundamentals", "sector": "I
"dbms": {"category": "CS Fundamentals", "sector": "IT", "demand":
"software testing": {"category": "QA", "sector": "IT", "demand":
"selenium": {"category": "QA", "sector": "IT", "demand": "Medium"
"api testing": {"category": "QA", "sector": "IT", "demand": "Medi
"unit testing": {"category": "QA", "sector": "IT", "demand": "Med

```

Soft Skills

```

"communication": {"category": "Soft Skills", "sector": "General",
"teamwork": {"category": "Soft Skills", "sector": "General", "dem
"leadership": {"category": "Soft Skills", "sector": "General", "d
"problem solving": {"category": "Soft Skills", "sector": "General
"critical thinking": {"category": "Soft Skills", "sector": "Gener
"project management": {"category": "Soft Skills", "sector": "Gene
"time management": {"category": "Soft Skills", "sector": "General

```

Mechanical Engineering

```

"autocad": {"category": "CAD", "sector": "Mechanical", "demand":
"solidworks": {"category": "CAD", "sector": "Mechanical", "demand
"catia": {"category": "CAD", "sector": "Mechanical", "demand": "M
"ansys": {"category": "Simulation", "sector": "Mechanical", "dema
"matlab": {"category": "Simulation", "sector": "Mechanical", "dem
"3d printing": {"category": "Manufacturing", "sector": "Mechanica

```

```

"cnc programming": {"category": "Manufacturing", "sector": "Mechanical", "demand": "High"},
"plc programming": {"category": "Automation", "sector": "Mechanical", "demand": "High"},
"thermodynamics": {"category": "Core", "sector": "Mechanical", "demand": "High"},
"fluid mechanics": {"category": "Core", "sector": "Mechanical", "demand": "High"},
"supply chain management": {"category": "Management", "sector": "Mechanical", "demand": "High"}

```

Civil Engineering

```

"staad pro": {"category": "Structural", "sector": "Civil", "demand": "High"},
"revit": {"category": "BIM", "sector": "Civil", "demand": "High"},
"bim": {"category": "BIM", "sector": "Civil", "demand": "High"},
"gis": {"category": "Geospatial", "sector": "Civil", "demand": "High"},
"primavera": {"category": "Project Management", "sector": "Civil", "demand": "High"},
"etabs": {"category": "Structural", "sector": "Civil", "demand": "High"},
"green building": {"category": "Sustainability", "sector": "Civil", "demand": "High"}

```

Electrical & Electronics

```

"embedded systems": {"category": "Embedded", "sector": "Electronics", "demand": "High"},
"verilog": {"category": "VLSI", "sector": "Electronics", "demand": "High"},
"vhdl": {"category": "VLSI", "sector": "Electronics", "demand": "High"},
"pcb design": {"category": "Hardware", "sector": "Electronics", "demand": "High"},
"power electronics": {"category": "Core", "sector": "Electronics", "demand": "High"},
"signal processing": {"category": "Core", "sector": "Electronics", "demand": "High"},
"scada": {"category": "Automation", "sector": "Electronics", "demand": "High"}

```

Management / Business

```

"digital marketing": {"category": "Marketing", "sector": "Management", "demand": "High"},
"seo": {"category": "Marketing", "sector": "Management", "demand": "High"},
"financial analysis": {"category": "Finance", "sector": "Management", "demand": "High"},
"business analytics": {"category": "Analytics", "sector": "Management", "demand": "High"},
"erp": {"category": "Enterprise", "sector": "Management", "demand": "High"},
"sap": {"category": "Enterprise", "sector": "Management", "demand": "High"},
"excel": {"category": "Tools", "sector": "Management", "demand": "High"},
"tally": {"category": "Accounting", "sector": "Management", "demand": "High"},

# Healthcare
"bioinformatics": {"category": "Healthcare IT", "sector": "Healthcare", "demand": "High"},
"medical imaging": {"category": "Healthcare IT", "sector": "Healthcare", "demand": "High"},
"health informatics": {"category": "Healthcare IT", "sector": "Healthcare", "demand": "High"},
"telemedicine": {"category": "Healthcare IT", "sector": "Healthcare", "demand": "High"},
}

# OBSOLETE SKILLS that should be flagged in syllabuses
OBSOLETE_SKILLS = {
    "cobol", "fortran", "turbo c", "turbo c++", "pascal", "visual basic",
    "vb.net", "foxpro", "dbase", "clipper", "lotus 1-2-3", "wordstar",
    "basic programming", "assembly language", "mainframe", "punch cards",
    "floppy disk", "dos programming", "windows xp", "dreamweaver",
    "flash", "actionscript", "silverlight", "coreldraw", "frontpage",
    "cold fusion", "perl", "cgi programming"
}

```

```
#    COURSE RECOMMENDATIONS for missing skills
```

```
COURSE_RECOMMENDATIONS = {
```

```
    "python": {"course": "Python for Everybody", "provider": "Coursera"},
    "java": {"course": "Java Programming Masterclass", "provider": "NPTEL"},
    "javascript": {"course": "The Complete JavaScript Course", "provider": "Udemy"},
    "react": {"course": "React - The Complete Guide", "provider": "Udemy"},
    "node.js": {"course": "Node.js API Masterclass", "provider": "NPTEL"},
    "machine learning": {"course": "Machine Learning by Andrew Ng", "provider": "Coursera"},
    "deep learning": {"course": "Deep Learning Specialization", "provider": "Coursera"},
    "aws": {"course": "AWS Cloud Practitioner", "provider": "AWS Training"},
    "docker": {"course": "Docker & Kubernetes Complete Guide", "provider": "Udemy"},
    "kubernetes": {"course": "Kubernetes for Developers", "provider": "Udemy"},
    "sql": {"course": "Complete SQL Bootcamp", "provider": "NPTEL / K"},
    "mongodb": {"course": "MongoDB University M001", "provider": "MongoDB"},
    "git": {"course": "Git & GitHub Crash Course", "provider": "freeCodeCamp"},
    "cybersecurity": {"course": "Intro to Cybersecurity", "provider": "Udemy"},
    "data structures": {"course": "Data Structures & Algorithms", "provider": "Udemy"},
    "generative ai": {"course": "Generative AI with LLMs", "provider": "Udemy"},
    "flutter": {"course": "Flutter & Dart Complete Guide", "provider": "Udemy"},
    "digital marketing": {"course": "Google Digital Marketing", "provider": "Udemy"},
    "power bi": {"course": "Power BI Data Analyst", "provider": "Microsoft"},
    "devops": {"course": "DevOps Engineering Course", "provider": "NPTEL"},
    "terraform": {"course": "Terraform Associate Certification", "provider": "Udemy"},
    "system design": {"course": "System Design for Interviews", "provider": "Udemy"},
```

```

    "agile": {"course": "Agile with Atlassian Jira", "provider": "Coursera"},
    "autocad": {"course": "AutoCAD Essentials Training", "provider": "Udacity"},
    "embedded systems": {"course": "Embedded Systems Design", "provider": "Udacity"},
    "blockchain": {"course": "Blockchain Basics", "provider": "Coursera"},
}

```

```

def extract_skills(text):
    """
    Extract skills from text using multi-word and single-word matching
    against the 200+ skill taxonomy.
    """
    text_lower = text.lower()

    # Handle common aliases to make matching more robust (as requested)
    aliases = {
        "powerbi": "power bi",
        "power-bi": "power bi",
        "mysql": "sql",
        "postgresql": "sql",
        "reactjs": "react",
        "nodejs": "node.js"
    }

    for alias, canonical in aliases.items():
        # Replace occurrences as a quick normalization

```

```

        text_lower = text_lower.replace(alias, canonical)

found = {}

# Sort skills by length (longest first) to match multi-word skills
sorted_skills = sorted(SKILL_TAXONOMY.keys(), key=len, reverse=True)

for skill in sorted_skills:
    # Use word boundary matching for accurate detection
    pattern = r'\b' + re.escape(skill) + r'\b'
    if re.search(pattern, text_lower):
        info = SKILL_TAXONOMY[skill]
        found[skill] = {
            "category": info["category"],
            "sector": info["sector"],
            "demand": info["demand"],
            "trend": info["trend"]
        }

return found

```

```

def detect_obsolete(text):
    """Detect obsolete/outdated skills mentioned in syllabus text."""
    text_lower = text.lower()

```

```

found = []
for skill in OBSOLETE_SKILLS:
    pattern = r'\b' + re.escape(skill) + r'\b'
    if re.search(pattern, text_lower):
        found.append(skill)
return found

def get_recommendations(missing_skills):
    """Generate course recommendations for each missing skill."""
    recs = []
    for skill in missing_skills:
        if skill in COURSE_RECOMMENDATIONS:
            rec = COURSE_RECOMMENDATIONS[skill].copy()
            rec["skill"] = skill
            recs.append(rec)
        else:
            recs.append({
                "skill": skill,
                "course": f"Industry Certification in {skill.title()}",
                "provider": "NPTEL / Coursera / Udemy",
                "duration": "6-8 weeks",
                "level": "Intermediate"
            })
    return recs

```

```

def calculate_alignment_score(matched, missing):
    """Calculate overall curriculum-industry alignment percentage."""
    total = len(matched) + len(missing)
    if total == 0:
        return 100
    return round((len(matched) / total) * 100)

def get_proficiency_level(skill_name, context_text):
    """Determine proficiency level based on job context keywords."""
    text_lower = context_text.lower()
    advanced_keywords = ["senior", "lead", "architect", "expert", "pr
    beginner_keywords = ["junior", "intern", "fresher", "entry level"

    for kw in advanced_keywords:
        if kw in text_lower:
            return "Advanced"
    for kw in beginner_keywords:
        if kw in text_lower:
            return "Beginner"
    return "Intermediate"

```

```

#     MAIN ANALYSIS ENDPOINT

@app.route('/analyze', methods=['POST'])
def analyze_gap():
    data = request.json

    if not data or 'syllabusText' not in data or 'jobText' not in data:
        return jsonify({'error': 'Missing syllabusText or jobText'}), 400

    syllabus_text = data['syllabusText']
    job_text = data['jobText']
    sector_filter = data.get('sector', None)

    # Extract skills from both texts
    syllabus_skills = extract_skills(syllabus_text)
    job_skills = extract_skills(job_text)

    # If sector filter is provided, only consider job skills from that sector
    if sector_filter and sector_filter != 'All':
        job_skills = {k: v for k, v in job_skills.items()
                       if v['sector'] == sector_filter or v['sector'] == 'All'}

    # Calculate matched and missing
    matched = {k: v for k, v in job_skills.items() if k in syllabus_skills}
    missing = {k: v for k, v in job_skills.items() if k not in syllabus_skills}

    # Detect obsolete skills in syllabus

```

```

obsolete = detect_obsolete(syllabus_text)

# Get proficiency levels for missing skills
missing_with_proficiency = []
for skill, info in missing.items():
    missing_with_proficiency.append({
        "name": skill,
        "category": info["category"],
        "demand": info["demand"],
        "trend": info["trend"],
        "proficiency": get_proficiency_level(skill, job_text)
    })

# Sort missing skills: High demand first, then Rising trend
demand_order = {"High": 0, "Medium": 1, "Low": 2}
missing_with_proficiency.sort(key=lambda x: (demand_order.get(x["demand"], 3)))

# Generate recommendations
recommendations = get_recommendations(list(missing.keys()))

# Alignment score
alignment_score = calculate_alignment_score(matched, missing)

# Demand analysis by category
demand_analysis = {}

```

```

for skill, info in job_skills.items():
    cat = info["category"]
    if cat not in demand_analysis:
        demand_analysis[cat] = {"total": 0, "matched": 0, "missing": 0}
    demand_analysis[cat]["total"] += 1
    if skill in matched:
        demand_analysis[cat]["matched"] += 1
    else:
        demand_analysis[cat]["missing"] += 1

return jsonify({
    'matched_skills': list(matched.keys()),
    'missing_skills': [s["name"] for s in missing_with_proficiency],
    'missing_skills_detailed': missing_with_proficiency,
    'obsolete_skills': obsolete,
    'recommendations': recommendations,
    'alignment_score': alignment_score,
    'demand_analysis': demand_analysis,
    'total_job_skills': len(job_skills),
    'total_syllabus_skills': len(syllabus_skills)
})

```

```

# SKILL TAXONOMY ENDPOINT

```

```

@app.route('/taxonomy', methods=['GET'])

```

```

def get_taxonomy():
    """Return full skill taxonomy for frontend display."""
    taxonomy = []
    for name, info in SKILL_TAXONOMY.items():
        taxonomy.append({
            "name": name,
            "category": info["category"],
            "sector": info["sector"],
            "demand": info["demand"],
            "trend": info["trend"]
        })
    return jsonify(taxonomy)

# TREND ANALYSIS ENDPOINT
@app.route('/trends', methods=['GET'])
def get_trends():
    """Return skill trend analysis - rising, stable, declining counts
    rising = [k for k, v in SKILL_TAXONOMY.items() if v["trend"] == "
    stable = [k for k, v in SKILL_TAXONOMY.items() if v["trend"] == "
    declining = [k for k, v in SKILL_TAXONOMY.items() if v["trend"] == "

    # Group rising skills by category
    rising_by_category = {}
    for skill in rising:

```

```

        cat = SKILL_TAXONOMY[skill]["category"]
        if cat not in rising_by_category:
            rising_by_category[cat] = []
        rising_by_category[cat].append(skill)

    return jsonify({
        'rising': rising,
        'stable': stable,
        'declining': declining,
        'rising_count': len(rising),
        'stable_count': len(stable),
        'declining_count': len(declining),
        'rising_by_category': rising_by_category,
        'top_emerging': rising[:10]
    })

```

```

#    DISTRICT ANALYSIS ENDPOINT

```

```

@app.route('/district-analysis', methods=['POST'])

```

```

def district_analysis():

```

```

    """Analyze skill demand for a specific district based on job post

```

```

    data = request.json

```

```

    if not data or 'jobs' not in data:

```

```

        return jsonify({'error': 'Missing jobs data'}), 400

```

```

district = data.get('district', 'All')
jobs = data['jobs']

# Aggregate all skills from jobs in this district
all_skills = {}
for job in jobs:
    if district == 'All' or job.get('location', '').lower() == di
        skills = extract_skills(job.get('requirements', ''))
        for skill, info in skills.items():
            if skill not in all_skills:
                all_skills[skill] = {"count": 0, **info}
            all_skills[skill]["count"] += 1

# Sort by count (most demanded first)
sorted_skills = sorted(all_skills.items(), key=lambda x: x[1]["co

return jsonify({
    'district': district,
    'top_skills': [{"name": k, **v} for k, v in sorted_skills[:20]
    'total_unique_skills': len(all_skills),
    'total_jobs_analyzed': len(jobs)
})

```

```
# GENERATE TRAINING PLAN
```

```

@app.route('/generate-training-plan', methods=['POST'])
def generate_training_plan():
    """Generate a district-level training plan based on skill gaps."""
    data = request.json
    target_skills = data.get('targetSkills', [])
    district = data.get('district', 'Unknown')

    courses = []
    trainers_needed = set()
    equipment = set()

    for skill in target_skills:
        if skill in COURSE_RECOMMENDATIONS:
            rec = COURSE_RECOMMENDATIONS[skill].copy()
            rec["skill"] = skill
            courses.append(rec)

            info = SKILL_TAXONOMY.get(skill, {})
            cat = info.get("category", "General")
            trainers_needed.add(f"{cat} domain expert")

            if cat in ["Programming", "Web Development", "AI/ML", "Data S
                equipment.add("Computer Lab with Internet")
            elif cat in ["CAD", "Simulation"]:
                equipment.add(f"Workstations with {skill.title()} software")

```

```

elif cat in ["Embedded", "Hardware"]:
    equipment.add("Electronics Lab with microcontrollers")
elif cat in ["Cloud", "DevOps"]:
    equipment.add("Cloud sandbox accounts (AWS/Azure)")

# Oversupply simulation logic: For a real region, this would query
# the number of grads vs job postings. For now we detect common 1
oversupplied = []
legacy = ["c programming", "php", "tally", "manual testing"]
for leg in legacy:
    if leg not in target_skills:
        oversupplied.append(leg.title())

return jsonify({
    'district': district,
    'target_skills': target_skills,
    'recommended_courses': courses,
    'trainer_requirements': list(trainers_needed),
    'equipment_needed': list(equipment),
    'oversupplied_skills': oversupplied[:2], # Flag top 2 oversup
    'estimated_timeline': f"{max(3, len(target_skills) * 2)} mont
    'estimated_batch_size': 30
})

```

```

if __name__ == '__main__':
    print(f"[ML Engine] Loaded {len(SKILL_TAXONOMY)} skills in taxonomy")
    print(f"[ML Engine] Loaded {len(OBSOLETE_SKILLS)} obsolete skills")
    print(f"[ML Engine] Loaded {len(COURSE_RECOMMENDATIONS)} course recommendations")
    app.run(port=5001, debug=True)

```

Source Code: index.ts

```

import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import { PrismaClient } from '@prisma/client';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import axios from 'axios';

dotenv.config();

const app = express();
const prisma = new PrismaClient();
const PORT = process.env.PORT || 5000;
const JWT_SECRET = process.env.JWT_SECRET || 'supersecretkey';
const ML_ENGINE_URL = 'http://127.0.0.1:5001';

app.use(cors());

```

```

app.use(express.json());

// AUTH MIDDLEWARE
const authenticate = (req: any, res: any, next: any) => {
  const token = req.header('Authorization')?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Access denied' });
  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.user = decoded;
    next();
  } catch (error) {
    res.status(400).json({ message: 'Invalid token' });
  }
};

//
// AUTH ROUTES
//

app.post('/api/auth/register', async (req, res) => {
  const { name, email, password, role, location, organization } = req.body;
  try {
    const existingUser = await prisma.user.findUnique({ where: { email } });
    if (existingUser) {
      res.status(400).json({ message: 'User already exists' });
    }
  }
});

```

```

        return;
    }

    const hashedPassword = await bcrypt.hash(password, 10);
    const user = await prisma.user.create({
        data: { name, email, password: hashedPassword, role, location:
    });

    const token = jwt.sign({ id: user.id, role: user.role }, JWT_SECRET);
    res.json({ token, user: { id: user.id, name: user.name, role: user.role } });
} catch (error) {
    console.error("Register Error:", error);
    res.status(500).json({ message: 'Server error' });
}
});

app.post('/api/auth/login', async (req, res) => {
    const { email, password } = req.body;
    try {
        const user = await prisma.user.findUnique({ where: { email } });
        if (!user) { res.status(400).json({ message: 'Invalid credentials' }); }
        const isMatch = await bcrypt.compare(password, user.password);
        if (!isMatch) { res.status(400).json({ message: 'Invalid credentials' }); }
        const token = jwt.sign({ id: user.id, role: user.role }, JWT_SECRET);
        res.json({ token, user: { id: user.id, name: user.name, role: user.role } });
    } catch (error) {
        res.status(500).json({ message: 'Server error' });
    }
});

```

```

    }
  });

  //
  // JOB POSTING ROUTES (COMPANY)
  //

app.post('/api/jobs', authenticate, async (req: any, res: any) => {
  if (req.user.role !== 'COMPANY') return res.status(403).json({ message: 'Unauthorized' });

  const { title, description, requirements, sector, location, proficiencyLevel, minExperience, salaryRange } = req.body;

  try {
    const job = await prisma.jobPosting.create({
      data: {
        title, description, requirements,
        sector: sector || 'IT',
        location: location || '',
        proficiencyLevel: proficiencyLevel || 'Intermediate',
        minExperience: minExperience || 0,
        salaryRange: salaryRange || '',
        companyId: req.user.id
      }
    });
  } catch (error) {
    console.error("Job Post Error:", error);
  }
});

```

```

        res.status(500).json({ message: 'Server error' });
    }
});

```

```

app.get('/api/jobs', async (req, res) => {
    try {
        const jobs = await prisma.jobPosting.findMany({
            include: { company: { select: { name: true } } },
            orderBy: { createdAt: 'desc' }
        });
        res.json(jobs);
    } catch (error) {
        res.status(500).json({ message: 'Server error' });
    }
});

```

```

//
// SYLLABUS ROUTES (COLLEGE) + ML GAP ANALYSIS
//

```

```

app.post('/api/syllabus', authenticate, async (req: any, res: any) => {
    if (req.user.role !== 'COLLEGE') return res.status(403).json({ message: 'Unauthorized' });
    const { branch, semester, subjects, content, university, lastUpdate } = req.body;
    try {
        const syllabus = await prisma.syllabus.create({

```

```

    data: {
      branch, semester, subjects: subjects || 'Various', content,
      university: university || '',
      lastUpdated: lastUpdated || '2024',
      collegeId: req.user.id
    }
  });

// Fetch all job requirements for gap analysis
const jobs = await prisma.jobPosting.findMany();
const allJobRequirements = jobs.map(j => `${j.title}: ${j.requirements}`);

// Call Python ML Microservice
try {
  const mlResponse = await axios.post(`${ML_ENGINE_URL}/analyze`, {
    syllabusText: content,
    jobText: allJobRequirements
  });

  const mlData = mlResponse.data;

  const gapReport = await prisma.gapReport.create({
    data: {
      missingSkills: (mlData.missing_skills || []).join(', '),
      matchedSkills: (mlData.matched_skills || []).join(', '),
    }
  });
} catch (error) {
  console.error('Error calling ML Microservice:', error);
}

```

```

        obsoleteSkills: (mlData.obsolete_skills || []).join(', '),
        recommendations: JSON.stringify(mlData.recommendations || []),
        overallScore: mlData.alignment_score || 0,
        demandAnalysis: JSON.stringify(mlData.demand_analysis || {}),
        collegeId: req.user.id,
        syllabusId: syllabus.id
    }
  });

  res.json({ syllabus, gapReport, mlData });
} catch (mlError) {
  console.error('ML Engine Error:', mlError);
  // Still save syllabus even if ML fails
  res.json({ syllabus, gapReport: null, mlError: 'ML engine unavailable' });
}
} catch (error) {
  console.error("Syllabus Error:", error);
  res.status(500).json({ message: 'Server error' });
}
});

app.get('/api/syllabus', async (req, res) => {
  try {
    const syllabuses = await prisma.syllabus.findMany({
      include: { college: { select: { name: true } } },
      orderBy: { createdAt: 'desc' }
    });
  }
});

```

```

    });
    res.json(syllabuses);
  } catch (error) {
    res.status(500).json({ message: 'Server error' });
  }
});

//
// GAP REPORT ROUTES
//

app.get('/api/reports', async (req, res) => {
  try {
    const reports = await prisma.gapReport.findMany({
      include: {
        college: { select: { name: true, location: true } },
        syllabus: { select: { branch: true, semester: true, universit
      },
      orderBy: { createdAt: 'desc' }
    });
    res.json(reports);
  } catch (error) {
    res.status(500).json({ message: 'Server error' });
  }
});

```

```

// Employer validation of gap reports
app.post('/api/reports/:id/validate', authenticate, async (req: any, res: any) => {
  if (req.user.role !== 'COMPANY') return res.status(403).json({ message: 'Unauthorized' });
  const { id } = req.params;
  const { action, note } = req.body; // action = 'validate' | 'dispute'
  try {
    const report = await prisma.gapReport.findUnique({ where: { id } });
    if (!report) return res.status(404).json({ message: 'Report not found' });

    // Store validation info in demandAnalysis JSON field
    let demandData: any = {};
    try { demandData = JSON.parse(report.demandAnalysis || '{}'); } catch { }
    const validations: any[] = demandData.validations || [];
    validations.push({
      companyId: req.user.id,
      action,
      note: note || '',
      timestamp: new Date().toISOString()
    });
    demandData.validations = validations;
    demandData.validationStatus = action === 'validate' ? 'Employer Validated' : 'Disputed';

    const updated = await prisma.gapReport.update({
      where: { id },

```

```

        data: { demandAnalysis: JSON.stringify(demandData) }
    });
    res.json({ message: `Report ${action}d successfully`, status: dem
} catch (error) {
    console.error('Validate Error:', error);
    res.status(500).json({ message: 'Server error' });
}
});

```

```

// College marks a gap report as 'Revised' after updating curriculum
app.patch('/api/reports/:id/status', authenticate, async (req: any, r
    if (req.user.role !== 'COLLEGE') return res.status(403).json({ mess
    const { id } = req.params;
    const { status } = req.body; // e.g. 'Revised', 'Under Review'
    try {
        const report = await prisma.gapReport.findUnique({ where: { id }
        if (!report) return res.status(404).json({ message: 'Report not f
        // Store revision status inside demandAnalysis JSON
        let demandData: any = {};
        try { demandData = JSON.parse(report.demandAnalysis || '{}'); } c
        demandData.revisionStatus = status || 'Revised';
        demandData.revisedAt = new Date().toISOString();
        demandData.revisedBy = req.user.id;
        const updated = await prisma.gapReport.update({
            where: { id },

```

```

        data: { demandAnalysis: JSON.stringify(demandData) }
    });
    res.json({ message: 'Report marked as ' + (status || 'Revised') },
} catch (error) {
    console.error('Report Status Error:', error);
    res.status(500).json({ message: 'Server error' });
}
});

//
// EMPLOYER SURVEY ROUTES (COMPANY)
//

app.post('/api/surveys', authenticate, async (req: any, res: any) =>
    if (req.user.role !== 'COMPANY') return res.status(403).json({ mess
    const { satisfactionScore, feedbackText, skillsNeeded, hiringDistri
    try {
        const survey = await prisma.employerSurvey.create({
            data: {
                satisfactionScore: satisfactionScore || 3,
                feedbackText: feedbackText || '',
                skillsNeeded: skillsNeeded || '',
                hiringDistrict: hiringDistrict || '',
                candidateQuality: candidateQuality || 'Average',
                companyId: req.user.id

```

```

    }
  });
  res.json(survey);
} catch (error) {
  console.error("Survey Error:", error);
  res.status(500).json({ message: 'Server error' });
}
});

app.get('/api/surveys', async (req, res) => {
  try {
    const surveys = await prisma.employerSurvey.findMany({
      include: { company: { select: { name: true } } },
      orderBy: { createdAt: 'desc' }
    });
    res.json(surveys);
  } catch (error) {
    res.status(500).json({ message: 'Server error' });
  }
});

//
// INDUSTRY CONSULTATION ROUTES
//

```

```

// In-memory store for consultations (SQLite schema avoids migration
const consultations: any[] = [];

app.post('/api/consultations', authenticate, async (req: any, res: an
  if (req.user.role !== 'COMPANY') return res.status(403).json({ mess
  const { sector, district, requiredRoles, emergingSkills, timeline,
  if (!sector || !district || !requiredRoles) {
    return res.status(400).json({ message: 'sector, district, and req
  }
  const consultation = {
    id: `c-${Date.now()}`,
    companyId: req.user.id,
    sector,
    district,
    requiredRoles,
    emergingSkills: emergingSkills || '',
    timeline: timeline || '6 months',
    urgency: urgency || 'Medium',
    notes: notes || '',
    createdAt: new Date().toISOString()
  };
  consultations.push(consultation);
  res.status(201).json(consultation);
});

```

```

app.get('/api/consultations', async (req, res) => {
  res.json(consultations);
});

app.get('/api/surveys/stats', async (req, res) => {
  try {
    const surveys = await prisma.employerSurvey.findMany();
    const total = surveys.length;
    const avgSatisfaction = total > 0
      ? (surveys.reduce((sum, s) => sum + s.satisfactionScore, 0) / total)
      : 0;

    // Aggregate skills needed
    const skillCounts: Record<string, number> = {};
    surveys.forEach(s => {
      s.skillsNeeded.split(',').forEach(skill => {
        const trimmed = skill.trim().toLowerCase();
        if (trimmed) skillCounts[trimmed] = (skillCounts[trimmed] || 0) + 1;
      });
    });

    const topNeededSkills = Object.entries(skillCounts)
      .sort(([, a], [, b]) => b - a)
      .slice(0, 10)
      .map(([skill, count]) => ({ skill, count }));
  } catch (error) {
    res.status(500).json({ error: 'Failed to fetch survey statistics' });
  }
});

```

```

    // Quality distribution
    const qualityDist: Record<string, number> = {};
    surveys.forEach(s => {
        qualityDist[s.candidateQuality] = (qualityDist[s.candidateQuali
    });

    res.json({ total, avgSatisfaction, topNeededSkills, qualityDistri
} catch (error) {
    res.status(500).json({ message: 'Server error' });
}
});

//
// PLACEMENT TRACKING ROUTES
//

// Single correct placement route - accepts studentEmail + companyNam
app.post('/api/placements', authenticate, async (req: any, res: any)
    if (req.user?.role !== 'COLLEGE' && req.user?.role !== 'ADMIN') {
        return res.status(403).json({ message: 'Only colleges or admins c
    }
    const { studentEmail, companyName, jobRole, package: pkg, location
    if (!studentEmail || !companyName || !jobRole) {
        return res.status(400).json({ message: 'studentEmail, companyName
    }

```

```

try {
  // Find or create student by email
  let student = await prisma.user.findUnique({ where: { email: stud
  if (!student) {
    const hp = await bcrypt.hash('password123', 10);
    student = await prisma.user.create({
      data: { name: studentEmail.split('@')[0], email: studentEmail
    });
  }

  // Find or create company by name
  let company = await prisma.user.findFirst({ where: { name: compan
  if (!company) {
    const hp = await bcrypt.hash('password123', 10);
    company = await prisma.user.create({
      data: { name: companyName, email: `${companyName.replace(/\s+
    });
  }

  const placement = await prisma.placementRecord.create({
    data: {
      jobRole,
      package: pkg || '',
      location: location || '',
      isPlaced: true,
      studentId: student.id,
      companyId: company.id

```

```

    }
  });
  res.status(201).json({ ...placement, studentName: student.name, c
} catch (error) {
  console.error('Placement Error:', error);
  res.status(500).json({ message: 'Server error adding placement' }
}
});

```

```

app.get('/api/career-pathways', async (req, res) => {
  try {
    const jobs = await prisma.jobPosting.findMany();
    const roleStats: Record<string, { count: number, salaries: number

    jobs.forEach(job => {
      const role = job.title;
      if (!roleStats[role]) roleStats[role] = { count: 0, salaries: []
      roleStats[role].count++;

      // Parse salary e.g. "6-10 LPA" -> 8
      const match = job.salaryRange.match(/(\d+)/g);
      if (match && match.length > 0) {
        const avg = match.reduce((a, b) => a + parseInt(b), 0) / match
        roleStats[role].salaries.push(avg);
      }
    }
  }
}

```

```

    job.requirements.split(',').forEach(s => {
      const skill = s.trim();
      if (skill) roleStats[role].skills[skill] = (roleStats[role].s
    });
  });

const pathways = Object.entries(roleStats)
  .filter(([_, stats]) => stats.count > 0)
  .map(([title, stats]) => {
    const avgSalary = stats.salaries.length > 0
      ? `${Math.round(stats.salaries.reduce((a, b) => a + b, 0) /
      : 'Competitive';

const topSkills = Object.entries(stats.skills)
  .sort((a, b) => b[1] - a[1])
  .slice(0, 5)
  .map(s => s[0]);

return {
  title,
  demand: stats.count > 2 ? 'Very High' : 'High',
  salary: avgSalary,
  skills: topSkills,
  emoji: title.toLowerCase().includes('data') ? '' : title.t

```

```

        };
    })
    .sort((a, b) => (b.demand === 'Very High' ? 1 : 0) - (a.demand

    res.json(pathways);
} catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Server error' });
}
});

app.get('/api/placements', async (req, res) => {
    try {
        const placements = await prisma.placementRecord.findMany({
            include: {
                student: { select: { name: true } },
                company: { select: { name: true } }
            },
            orderBy: { createdAt: 'desc' }
        });
        res.json(placements);
    } catch (error) {
        res.status(500).json({ message: 'Server error' });
    }
});

```

```

app.get('/api/placements/stats', async (req, res) => {
  try {
    const placements = await prisma.placementRecord.findMany();
    const totalPlaced = placements.filter(p => p.isPlaced).length;
    const totalStudents = await prisma.user.count({ where: { role: 'S

// Location-wise breakdown
const locationDist: Record<string, number> = {};
placements.forEach(p => {
  const loc = p.location || 'Unknown';
  locationDist[loc] = (locationDist[loc] || 0) + 1;
});

// Role-wise breakdown
const roleDist: Record<string, number> = {};
placements.forEach(p => {
  roleDist[p.jobRole] = (roleDist[p.jobRole] || 0) + 1;
});

res.json({
  totalPlaced,
  totalStudents,
  placementRate: totalStudents > 0 ? ((totalPlaced / totalStudent
  locationDistribution: locationDist,

```

```

        roleDistribution: roleDist
    });
} catch (error) {
    res.status(500).json({ message: 'Server error' });
}
});

//
// SKILL TAXONOMY & TRENDS (Proxy to ML Engine)
//

app.get('/api/skills/taxonomy', async (req, res) => {
    try {
        const response = await axios.get(`${ML_ENGINE_URL}/taxonomy`);
        res.json(response.data);
    } catch (error) {
        res.status(500).json({ message: 'ML Engine unavailable' });
    }
});

app.get('/api/skills/trends', async (req, res) => {
    try {
        const response = await axios.get(`${ML_ENGINE_URL}/trends`);
        res.json(response.data);
    } catch (error) {

```

```

        res.status(500).json({ message: 'ML Engine unavailable' });
    }
});

//
// TRAINING PLAN ROUTES
//

app.post('/api/training-plans', async (req: any, res: any) => {
    const { district, targetSkills } = req.body;
    try {
        const mlResponse = await axios.post(`${ML_ENGINE_URL}/generate-tr
            district, targetSkills
        });
        const plan = mlResponse.data;

        // Store equipment + oversupply as JSON so frontend can parse bot
        const equipmentJson = JSON.stringify({
            equipment: plan.equipment_needed,
            oversupplied: plan.oversupplied_skills || []
        });

        const saved = await prisma.trainingPlan.create({
            data: {
                district,

```

```

        targetSkills: targetSkills.join(', '),
        recommendedCourses: JSON.stringify(plan.recommended_courses),
        trainerRequirements: plan.trainer_requirements.join(', '),
        equipmentNeeded: equipmentJson,
        timeline: plan.estimated_timeline
    }
});

res.json({ ...saved, details: plan });
} catch (error) {
    console.error("Training Plan Error:", error);
    res.status(500).json({ message: 'Server error generating training
    }
});

app.get('/api/training-plans', async (req, res) => {
    try {
        const plans = await prisma.trainingPlan.findMany({ orderBy: { cre
        res.json(plans);
    } catch (error) {
        res.status(500).json({ message: 'Server error' });
    }
});

//

```

```

// ANALYTICS DASHBOARD
//

app.get('/api/analytics/dashboard', async (req, res) => {
  try {
    const [totalUsers, totalJobs, totalReports, totalPlacements, totalEmployerSurveys, totalTrainingPlans] = await Promise.all([
      prisma.user.count(),
      prisma.jobPosting.count(),
      prisma.gapReport.count(),
      prisma.placementRecord.count(),
      prisma.employerSurvey.count(),
      prisma.trainingPlan.count()
    ]);

    const roleBreakdown = {
      students: await prisma.user.count({ where: { role: 'STUDENT' } }),
      colleges: await prisma.user.count({ where: { role: 'COLLEGE' } }),
      companies: await prisma.user.count({ where: { role: 'COMPANY' } })
    };

    // Average alignment score
    const reports = await prisma.gapReport.findMany({ select: { overallScore: true } });
    const avgAlignment = reports.length > 0
      ? (reports.reduce((sum, r) => sum + r.overallScore, 0) / reports.length)
      : 0;
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Internal server error' });
  }
});

```

```

// Top missing skills across all reports
const allReports = await prisma.gapReport.findMany({ select: { mi
const skillFreq: Record<string, number> = {};
allReports.forEach(r => {
  r.missingSkills.split(',').forEach(s => {
    const trimmed = s.trim().toLowerCase();
    if (trimmed) skillFreq[trimmed] = (skillFreq[trimmed] || 0) +
  });
});
const topMissingSkills = Object.entries(skillFreq)
  .sort(([a], [b]) => b - a)
  .slice(0, 10)
  .map(([skill, count]) => ({ skill, count }));

// Sector growth breakdown - aggregate job postings by sector
const allJobs = await prisma.jobPosting.findMany({ select: { sect
const sectorCounts: Record<string, number> = {};
const proficiencyCounts: Record<string, number> = {};
allJobs.forEach(j => {
  sectorCounts[j.sector] = (sectorCounts[j.sector] || 0) + 1;
  proficiencyCounts[j.proficiencyLevel] = (proficiencyCounts[j.pr
});
const sectorGrowth = Object.entries(sectorCounts)
  .map(([sector, count]) => ({ sector, count }));

```

```

        .sort((a, b) => b.count - a.count);
const proficiencyDemand = Object.entries(proficiencyCounts)
    .map(([level, count]) => ({ level, count }))
    .sort((a, b) => b.count - a.count);

// Count employer validations
const allGapReports = await prisma.gapReport.findMany({ select: {
let validatedCount = 0, revisedCount = 0;
allGapReports.forEach(r => {
    try {
        const d = JSON.parse(r.demandAnalysis || '{}');
        if (d.validationStatus === 'Employer Validated') validatedCount++;
        if (d.revisionStatus === 'Revised') revisedCount++;
    } catch(e) {}
});

res.json({
    totalUsers, totalJobs, totalReports, totalPlacements, totalSurveys,
    roleBreakdown, avgAlignment, topMissingSkills,
    sectorGrowth, proficiencyDemand, validatedCount, revisedCount
});
} catch (error) {
    console.error("Dashboard Error:", error);
    res.status(500).json({ message: 'Server error' });
}

```

```
});
```

```
app.get('/api/analytics/district/:district', async (req, res) => {  
  const { district } = req.params;  
  try {  
    const jobs = await prisma.jobPosting.findMany({  
      where: district !== 'all' ? { location: { contains: district }  
    });
```

```
    // Send to ML engine for district analysis
```

```
    const mlResponse = await axios.post(`${ML_ENGINE_URL}/district-an  
      district,  
      jobs: jobs.map(j => ({ requirements: j.requirements, location:
```

```
    res.json(mlResponse.data);
```

```
  } catch (error) {
```

```
    res.status(500).json({ message: 'Server error' });
```

```
  }
```

```
});
```

```
//
```

```
// SEED ENDPOINT (for demo data)
```

```
//
```

```

app.post('/api/seed', async (req, res) => {
  try {
    // Create demo companies

    const hp = await bcrypt.hash('demo123', 10);
    const company1 = await prisma.user.upsert({
      where: { email: 'tcs@demo.com' },
      update: {},
      create: { name: 'TCS', email: 'tcs@demo.com', password: hp, rol
    });

    const company2 = await prisma.user.upsert({
      where: { email: 'infosys@demo.com' },
      update: {},
      create: { name: 'Infosys', email: 'infosys@demo.com', password:
    });

    const company3 = await prisma.user.upsert({
      where: { email: 'wipro@demo.com' },
      update: {},
      create: { name: 'Wipro', email: 'wipro@demo.com', password: hp,
    });

    // Create demo admin

    await prisma.user.upsert({
      where: { email: 'admin@demo.com' },
      update: {},
      create: { name: 'Super Admin', email: 'admin@demo.com', passwor

```

```

});

// Create demo college
await prisma.user.upsert({
  where: { email: 'jiet@demo.com' },
  update: {},
  create: { name: 'JIET College', email: 'jiet@demo.com', password: '12345678' }
});

// Create demo student
await prisma.user.upsert({
  where: { email: 'student@demo.com' },
  update: {},
  create: { name: 'Demo Student', email: 'student@demo.com', password: '12345678' }
});

// Create demo jobs
const jobsData = [
  { title: 'Full Stack Developer', description: 'Build web applications' },
  { title: 'Data Scientist', description: 'Analyze large datasets' },
  { title: 'Cloud Engineer', description: 'Manage cloud infrastructure' },
  { title: 'Frontend Developer', description: 'Build responsive user interfaces' },
  { title: 'DevOps Engineer', description: 'Automate deployment pipelines' },
  { title: 'Cybersecurity Analyst', description: 'Protect digital assets' }
];

```

```

    for (const job of jobsData) {
      await prisma.jobPosting.create({ data: job }).catch(() => {});
    }

    // Create demo surveys
    const surveysData = [
      { satisfactionScore: 3, feedbackText: 'Students lack practical'
      { satisfactionScore: 2, feedbackText: 'Need more ML/AI trained'
      { satisfactionScore: 4, feedbackText: 'Good fundamentals but we
    ];
    for (const survey of surveysData) {
      await prisma.employerSurvey.create({ data: survey }).catch(() =
    }

    res.json({ message: 'Demo data seeded successfully!' });
  } catch (error) {
    console.error("Seed Error:", error);
    res.status(500).json({ message: 'Seed error', error });
  }
});

//   START SERVER
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
  console.log(`ML Engine expected at ${ML_ENGINE_URL}`);

```

```
});
```

Source Code: CollegeDashboard.jsx

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';
import API from '../api';

export default function CollegeDashboard() {
  const [activeTab, setActiveTab] = useState('upload');
  const [reports, setReports] = useState([]);
  const [isAnalyzing, setIsAnalyzing] = useState(false);
  const [trainingPlans, setTrainingPlans] = useState([]);
  const navigate = useNavigate();
  const token = localStorage.getItem('token');

  // Syllabus form
  const [form, setForm] = useState({ branch: '', semester: '', subject: '' });

  // Training plan form
  const [tpForm, setTpForm] = useState({ district: '', targetSkills: '' });

  // Placement form
  const [placementForm, setPlacementForm] = useState({ studentEmail: '' });
```

```

const [placementMsg, setPlacementMsg] = useState('');

useEffect(() => {
  if (!token) navigate('/login');
  fetchReports();
  fetchPlans();
}, []);

const fetchReports = async () => {
  try { const r = await axios.get(`${API}/api/reports`); setReports
};

const fetchPlans = async () => {
  try { const r = await axios.get(`${API}/api/training-plans`); set
};

const handleUpload = async (e) => {
  e.preventDefault();
  setIsAnalyzing(true);
  try {
    const res = await axios.post(`${API}/api/syllabus`, form, { hea
    setForm({ branch: '', semester: '', subjects: '', content: '',
    fetchReports();
  } catch (err) {
    alert('Failed to analyze. Make sure ML engine is running.');
```

```

        setIsAnalyzing(false);
    }
};

const handleGeneratePlan = async (e) => {
    e.preventDefault();
    try {
        const skills = tpForm.targetSkills.split(',').map(s => s.trim());
        await axios.post(`${API}/api/training-plans`, { district: tpForm.district, skills });
        setTpForm({ district: '', targetSkills: '' });
        fetchPlans();
    } catch (err) {
        alert('Failed to generate plan');
    }
};

```

```

const handlePlacementSubmit = async (e) => {
    e.preventDefault();
    try {
        await axios.post(`${API}/api/placements`, placementForm, { headers: { 'Content-Type': 'application/json' } });
        setPlacementMsg('Placement record submitted successfully!');
        setPlacementForm({ studentEmail: '', companyName: '', jobRole: '' });
        setTimeout(() => setPlacementMsg(''), 3000);
    } catch (err) {
        alert('Failed to submit placement data.');
```

```

    }
};

return (
  <div className="dashboard-container animate-fade-in">
    <div className="dashboard-header">
      <h1> Neural College Hub</h1>
      <button className="btn-outline" onClick={() => { localStorage
    </div>

    <div className="tabs">
      <button className={`tab ${activeTab === 'upload' ? 'active' :
      <button className={`tab ${activeTab === 'reports' ? 'active'
      <button className={`tab ${activeTab === 'plans' ? 'active' :
      <button className={`tab ${activeTab === 'placements' ? 'activ
    </div>

    {placementMsg && <div style={{ background: 'rgba(16, 185, 129,

    {/*    UPLOAD SYLLABUS TAB    */}

    {activeTab === 'upload' && (
      <div className="dashboard-grid grid-2">
        <div className="glass-panel">
          <h2 style={{ marginBottom: '1rem', color: 'var(--accent-s

```

```

<p style={{ color: 'var(--text-muted)', marginBottom: '1rem' }}>
    Paste your syllabus content below. Our AI engine will c
</p>
<form onSubmit={handleUpload}>
    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr' }}>
        <input placeholder="Branch (e.g. Computer Science)" value={form.branch}>
        <input placeholder="Semester (e.g. 5)" value={form.semester}>
    </div>
    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr' }}>
        <input placeholder="University" value={form.university}>
        <input placeholder="Last Updated Year" value={form.lastUpdatedYear}>
    </div>
    <input placeholder="Subjects (comma-separated)" value={form.subjects}>
    <textarea placeholder="Paste full syllabus content here" value={form.syllabus}>
    <button type="submit" className="btn-success" style={{
        display: 'flex', align-items: center, justify-content: center, gap: 10px }}>
        {isAnalyzing ? 'Scanning Matrices...' : 'Run AI Analysis'}
    </button>
</form>
</div>
<div className="glass-panel">
    <h2 style={{ marginBottom: '1rem' }}>Quick Stats</h2>
    <div className="dashboard-grid grid-2" style={{ marginBot
    <div className="stat-card">
        <div className="stat-value">{reports.length}</div>
        <div className="stat-label">Reports Generated</div>

```

```

    </div>
    <div className="stat-card">
      <div className="stat-value">{reports.length > 0 ? Mat
      <div className="stat-label">Avg Alignment</div>
    </div>
  </div>
  {reports.length > 0 && (
    <div>
      <h3 style={{ marginBottom: '0.75rem', fontSize: '0.95
      <ReportCard report={reports[0]} compact />
    </div>
  )}
</div>
</div>
)}

{/*    GAP REPORTS TAB    */}
{activeTab === 'reports' && (
  <div>
    {reports.length === 0 ? (
      <div className="glass-panel" style={{ textAlign: 'center'
        <p style={{ color: 'var(--text-muted)', fontSize: '1.1r
      </div>
    ) : (
      reports.map(r => <ReportCard key={r.id} report={r} />)

```

```

    })
  </div>
})

{/*    TRAINING PLANS TAB    */}
{activeTab === 'plans' && (
  <div className="dashboard-grid grid-2">
    <div className="glass-panel">
      <h2 style={{ marginBottom: '1rem', color: 'var(--secondary)' }}>
        <p style={{ color: 'var(--text-muted)', marginBottom: '1rem' }}>
          Enter a district and target skills to generate a comprehensive plan
        </p>
        <form onSubmit={handleGeneratePlan}>
          <input placeholder="District (e.g. Jodhpur)" value={tpForm.district} />
          <textarea placeholder="Target Skills (comma-separated, e.g. Agriculture, Healthcare)" value={tpForm.targetSkills} />
          <button type="submit" style={{ width: '100%' }}>Generate Plan</button>
        </form>
      </div>
      <div className="glass-panel" style={{ maxHeight: '600px', overflow: 'auto' }}>
        <h2 style={{ marginBottom: '1rem' }}>Generated Plans ({trainingPlans.length})
        {trainingPlans.length === 0 && <p style={{ color: 'var(--text-muted)' }}>No plans generated yet.</p>
        {trainingPlans.map(p => {
          let courses = [];
          try { courses = JSON.parse(p.recommendedCourses || '[]'); } catch { }
          // Parse equipment and oversupply from stored JSON

```

```

let equipment = p.equipmentNeeded;
let oversupplied = [];
try {
  const eq = JSON.parse(p.equipmentNeeded || '{}');
  if (eq.equipment) { equipment = eq.equipment.join(','); }
} catch(e) {}
return (
  <div key={p.id} style={{ background: 'rgba(0,0,0,0.2)' }}>
    <div style={{ display: 'flex', justifyContent: 'space-between' }}>
      <h3> {p.district}</h3>
      <span className="skill-tag rising">{p.status || ' '}</span>
    </div>

    <div style={{ marginBottom: '0.5rem' }}>
      <strong style={{ fontSize: '0.8rem', color: 'var(--text-muted)' }}>
        <div style={{ marginTop: '0.25rem' }}>
          {p.targetSkills.split(',').map(s => <span key={s}>{s}</span>)}
        </div>
      </div>

      <div style={{ marginBottom: '0.5rem', fontSize: '0.8rem' }}>
        <strong> AI Mentors Required: </strong>
        <span style={{ color: 'var(--text-muted)' }}>{p.targetMentors}</span>
      </div>

      <div style={{ marginBottom: '0.5rem', fontSize: '0.8rem' }}>

```

```

    <strong> Equipment: </strong>
    <span style={{ color: 'var(--text-muted)' }}>{equ
  </div>
<div style={{ marginBottom: '0.75rem', fontSize: '0
  <strong> Timeline: </strong>
  <span style={{ color: 'var(--accent-warning)' }}>
</div>

```

```

{courses.length > 0 && (
  <div style={{ marginBottom: '0.75rem' }}>
    <strong style={{ fontSize: '0.8rem', color: 'va
    {courses.map((c, i) => (
      <div key={i} style={{ background: 'rgba(99,10
        <strong>{c.course}</strong> - {c.provider}
      </div>
    )))}
  </div>
)}

```

```

{oversupplied.length > 0 && (
  <div style={{ background: 'rgba(245, 158, 11, 0.1
    <strong style={{ fontSize: '0.8rem', color: 'va
Do NOT invest training capacity here:</strong>
  <div style={{ marginTop: '0.25rem' }}>
    {oversupplied.map(s => <span key={s} classNam

```

```

        </div>
    </div>
    )}
</div>
);
}}
</div>
</div>
)}

```

```

{ /*    PLACEMENTS TAB    */
{activeTab === 'placements' && (
    <div className="dashboard-grid grid-2">
        <div className="glass-panel">
            <h2 style={{ marginBottom: '1rem', color: 'var(--primary)'>
            <p style={{ color: 'var(--text-muted)', marginBottom: '1r
                Report student placements to help build an evidence-bas
            </p>
            <form onSubmit={handlePlacementSubmit}>
                <input type="email" placeholder="Student Email (e.g. jo
                <input placeholder="Hiring Company (e.g. TCS, Infosys)"
                <input placeholder="Job Role (e.g. Full Stack Developer
                <div style={{ display: 'grid', gridTemplateColumns: '1f
                    <input placeholder="Package (e.g. 8 LPA)" value={plac
                    <input placeholder="Location (e.g. Pune)" value={plac

```

```

        </div>

        <button type="submit" style={{ width: '100%' }}>Submit
    </form>
</div>

<div className="glass-panel">
    <h2 style={{ marginBottom: '1rem' }}>Why track placements
    <ul style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
        <li><strong>Validate Curricula:</strong> Proves whether
        <li><strong>Track Demand:</strong> Shows which companies
        <li><strong>Better ROI:</strong> Demonstrates the return
        <li><strong>Career Guidance:</strong> Provides real data
    </ul>
</div>
</div>

    )}
</div>

);
}

```

```

function ReportCard({ report, compact, onRevised }) {
    const r = report;
    const token = localStorage.getItem('token');
    let recommendations = [];
    try { recommendations = JSON.parse(r.recommendations || '[]'); } catch
    let demandAnalysis = {};

```



```

<div>
    <h3 style={{ marginBottom: '0.2rem' }}>{r.college?.name} -
    {r.syllabus?.branch} (Sem {r.syllabus?.semester})</h3>
    {r.syllabus?.university && <span style={{ fontSize: '0.8rem' }}>
    <div style={{ display: 'flex', gap: '0.5rem', marginTop: '0.5rem' }}>
        {revisionStatus === 'Revised' && (
            <span className="skill-tag matched"> Curriculum Revised
        )}
        {validationStatus === 'Employer Validated' && (
            <span className="skill-tag matched"> Employer Validated
        )}
        {validationStatus === 'Employer Disputed' && (
            <span className="skill-tag missing"> Employer Disputed
        )}
    </div>
</div>
</div>
<div style={{ textAlign: 'right' }}>
    <div style={{ fontSize: '1.8rem', fontWeight: 800, color: 'var(--text-primary)' }}>
        <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)' }}>
    </div>
</div>
</div>

{/* Score Bar */}
<div className="score-bar-bg" style={{ marginBottom: '1.5rem' }}>
    <div className="score-bar-fill" style={{ width: `${r.overallScore}%` }}>

```

```
</div>
```

```
{/* Missing Skills */}
```

```
<div style={{ marginBottom: '1rem' }}>
```

```
  <h4 style={{ color: 'var(--accent-danger)', marginBottom: '0.25rem' }}>
```

```
    <div style={{ display: 'flex', flexWrap: 'wrap', gap: '0.25rem' }}>
```

```
      {r.missingSkills.split(',').filter(s => s.trim()).map(s =>
```

```
    </div>
```

```
</div>
```

```
{/* Matched Skills */}
```

```
<div style={{ marginBottom: '1rem' }}>
```

```
  <h4 style={{ color: 'var(--accent-success)', marginBottom: '0.25rem' }}>
```

```
    <div style={{ display: 'flex', flexWrap: 'wrap', gap: '0.25rem' }}>
```

```
      {r.matchedSkills.split(',').filter(s => s.trim()).map(s =>
```

```
    </div>
```

```
</div>
```

```
{/* Obsolete Skills */}
```

```
{r.obsoleteSkills && r.obsoleteSkills.trim() && (
```

```
  <div style={{ marginBottom: '1rem' }}>
```

```
    <h4 style={{ color: 'var(--accent-warning)', marginBottom: '0.25rem' }}>
```

```
    <div style={{ display: 'flex', flexWrap: 'wrap', gap: '0.25rem' }}>
```

```
      {r.obsoleteSkills.split(',').filter(s => s.trim()).map(s =>
```

```
    </div>
```

```

    </div>
  )}

  {/* Curriculum Recommendations */}
  {!compact && recommendations.length > 0 && (
    <div style={{ marginTop: '1.5rem' }}>
      <h4 style={{ marginBottom: '0.75rem', fontSize: '0.9rem' }}>
        <table className="data-table">
          <thead>
            <tr><th>Skill</th><th>Recommended Course</th><th>Provid
          </thead>
          <tbody>
            {recommendations.map((rec, i) => (
              <tr key={i}>
                <td style={{ fontWeight: 600 }}>{rec.skill}</td>
                <td>{rec.course}</td>
                <td style={{ color: 'var(--text-muted)' }}>{rec.pro
                <td>{rec.duration}</td>
                <td><span className="skill-tag rising">{rec.level}<
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    )}
  )}

```

```

    { /* Mark as Revised Action */ }
    { !compact && (
      <div style={{ marginTop: '1.5rem', paddingTop: '1rem', border
        <span style={{ fontSize: '0.85rem', color: 'var(--text-mute
          { revisionStatus === 'Revised'
            ? ' You have marked this curriculum as updated.'
            : 'After updating your curriculum based on these recomm
        </span>
        { revisionStatus !== 'Revised' && (
          <button
            className="btn-sm btn-success"
            onClick={handleMarkRevised}
            disabled={marking}
            style={{ whiteSpace: 'nowrap', marginLeft: '1rem' }}
          >
            { marking ? '...' : ' Mark Curriculum as Revised' }
          </button>
        ) }
      </div>
    ) }
  </div>
);
}

```

Source Code: CompanyDashboard.jsx

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';
import API from '../api';

const SECTORS = ['IT', 'Mechanical', 'Civil', 'Electronics', 'Managem
const LEVELS = ['Beginner', 'Intermediate', 'Advanced'];

export default function CompanyDashboard() {
  const [activeTab, setActiveTab] = useState('jobs');
  const [jobs, setJobs] = useState([]);
  const [message, setMessage] = useState('');
  const navigate = useNavigate();
  const token = localStorage.getItem('token');

  // Job form
  const [jobForm, setJobForm] = useState({ title: '', description: ''

  // Survey form
  const [surveyForm, setSurveyForm] = useState({ satisfactionScore: 3
  const [surveys, setSurveys] = useState([]);
```

```

// Industry Consultation form

const [consultForm, setConsultForm] = useState({ sector: 'IT', dist
const [consultations, setConsultations] = useState([]);
const [consultMsg, setConsultMsg] = useState('');

useEffect(() => {
  if (!token) navigate('/login');
  fetchJobs();
  fetchSurveys();
  fetchConsultations();
}, []);

const fetchJobs = async () => {
  try { const r = await axios.get(`${API}/api/jobs`); setJobs(r.dat
};

const fetchSurveys = async () => {
  try { const r = await axios.get(`${API}/api/surveys`); setSurveys
};

const fetchConsultations = async () => {
  try { const r = await axios.get(`${API}/api/consultations`); setC
};

const handlePostJob = async (e) => {
  e.preventDefault();
  try {

```

```

    // Append LPA if not present
    let formattedSalary = jobForm.salaryRange.trim();
    if (formattedSalary && !formattedSalary.toUpperCase().includes(
        formattedSalary += ' LPA';
    }

    const payload = {
        ...jobForm,
        requirements: jobForm.description, // Skills will automatical
        salaryRange: formattedSalary
    };

    await axios.post(`${API}/api/jobs`, payload, { headers: { Autho
    setMessage('Job posted successfully!');
    setTimeout(() => setMessage(''), 3000);
    setJobForm({ title: '', description: '', requirements: '', sect
    fetchJobs();
  } catch (err) { setMessage('Failed to post job'); }
};

const handleSubmitSurvey = async (e) => {
    e.preventDefault();
    try {
        await axios.post(`${API}/api/surveys`, surveyForm, { headers: {
        setMessage('Survey submitted! Thank you for your feedback.');
```

```

        setTimeout(() => setMessage(''), 3000);
        setSurveyForm({ satisfactionScore: 3, feedbackText: '', skillsM
        fetchSurveys();
    } catch (err) { setMessage('Failed to submit survey'); }
};

```

```

const handleSubmitConsultation = async (e) => {
    e.preventDefault();
    try {
        await axios.post(`${API}/api/consultations`, consultForm, { hea
        setConsultMsg('Industry consultation submitted! This data will
        setTimeout(() => setConsultMsg(''), 4000);
        setConsultForm({ sector: 'IT', district: '', requiredRoles: '',
        fetchConsultations();
    } catch (err) { setConsultMsg('Failed to submit: ' + (err.respons
};

```

```

return (
    <div className="dashboard-container animate-fade-in">
        <div className="dashboard-header">
            <h1> AI Employer Node</h1>
            <button className="btn-outline" onClick={() => { localStorage
        </div>

        {message && <div style={{ background: 'rgba(16, 185, 129, 0.15)

```

```

<div className="tabs">
  <button className={`tab ${activeTab === 'jobs' ? 'active' :
  <button className={`tab ${activeTab === 'consultation' ? 'act
  <button className={`tab ${activeTab === 'validate' ? 'active'
</div>

```

```

{ /*    POST JOBS TAB    */ }
{activeTab === 'jobs' && (
  <div className="dashboard-grid grid-2">
    <div className="glass-panel">
      <h2 style={{ marginBottom: '1rem', color: 'var(--accent-w
      <form onSubmit={handlePostJob}>
        <input placeholder="Job Title (e.g. Full Stack Develop
        <textarea placeholder="Job Description (Skills will be
        <div style={{ display: 'grid', gridTemplateColumns: '1f
          <select value={jobForm.sector} onChange={e => setJobF
            {SECTORS.map(s => <option key={s} value={s}>{s}</op
          </select>
          <select value={jobForm.proficiencyLevel} onChange={e
            {LEVELS.map(l => <option key={l} value={l}>{l}</opt
          </select>
        </div>
        <div style={{ display: 'grid', gridTemplateColumns: '1f
          <input placeholder="Location / District" value={jobFo

```

```

        <input placeholder="Salary (e.g. 6 or 4-6)" value={job.salaryRange} />
      </div>

      <button type="submit" className="btn-warning" style={{ width: 100px }}>
        Submit
      </button>
    </form>
  </div>

  <div className="glass-panel" style={{ maxHeight: '600px', overflow: 'auto' }}>
    <h2 style={{ marginBottom: '1rem' }}>All Job Postings ({jobs.length})
    </h2>
    {jobs.map(job => (
      <div key={job.id} style={{ background: 'rgba(0,0,0,0.2)', padding: '10px', margin: '10px 0' }}>
        <div style={{ display: 'flex', justifyContent: 'space-between' }}>
          <h3 style={{ color: 'var(--primary)' }}>{job.title}
          <span className="skill-tag rising">{job.sector}</span>
        </div>
        <details style={{ fontSize: '0.85rem', color: 'var(--text-secondary)' }}>
          <summary style={{ color: 'var(--primary)', outline: none }}>Details
          <p style={{ marginTop: '0.5rem', whiteSpace: 'pre-wrap' }}>
            {job.description}
          </p>
        </details>
        <div style={{ display: 'flex', gap: '1rem', fontSize: '0.85rem' }}>
          {job.location} && <span> {job.location}</span>
          <span> {job.proficiencyLevel}</span>
          {job.salaryRange} && <span> {job.salaryRange}</span>
        </div>
      </div>
    ))}
  </div>

```

```

    </div>
  })

```

```

{ /*    INDUSTRY CONSULTATION TAB    */
{activeTab === 'consultation' && (
  <div className="dashboard-grid grid-2">
    <div className="glass-panel">
      <h2 style={{ marginBottom: '0.5rem', color: 'var(--second
      <p style={{ color: 'var(--text-muted)', marginBottom: '1.
        Formally report your sector's upcoming skill demand so
it is a forward-looking demand signal for capacity planning.
      </p>
      {consultMsg && <div style={{ background: 'rgba(99,102,241
      <form onSubmit={handleSubmitConsultation}>
        <div style={{ display: 'grid', gridTemplateColumns: '1f
          <select value={consultForm.sector} onChange={e => set
            {SECTORS.map(s => <option key={s} value={s}>{s}</op
          </select>
          <input placeholder="District (e.g. Jodhpur, Pune)" va
        </div>
        <input placeholder="Roles you will need to hire for (e.
        <input placeholder="Emerging skills you expect to need
        <div style={{ display: 'grid', gridTemplateColumns: '1f
          <select value={consultForm.timeline} onChange={e => s

```

```

        <option value="3 months">Hiring in 3 months</option>
        <option value="6 months">Hiring in 6 months</option>
        <option value="12 months">Hiring in 12 months</option>
        <option value="2+ years">Long-term (2+ years)</option>
    </select>

    <select value={consultForm.urgency} onChange={e => setUrgency(e.target.value)}>
        <option value="Low">Low Priority</option>
        <option value="Medium">Medium Priority</option>
        <option value="High">High - Urgent Need</option>
    </select>
</div>

<textarea placeholder="Additional notes for training plan" />
<button type="submit" style={{ width: '100%', background-color: '#28a745', color: 'white', padding: '5px 10px', border: none, cursor: pointer }}>Submit</button>
</form>
</div>

<div className="glass-panel" style={{ maxHeight: '600px', overflow: 'auto', padding: '10px' }}>
    <h2 style={{ marginBottom: '1rem' }}>Recent Consultations</h2>
    {consultations.length === 0 ? (
        <p style={{ color: 'var(--text-muted)' }}>No consultations found</p>
    ) : consultations.slice().reverse().map(c => (
        <div key={c.id} style={{ background: 'rgba(0,0,0,0.2)', padding: '10px', margin: '5px 0', border: '1px solid #ccc' }}>
            <div style={{ display: 'flex', justifyContent: 'space-between' }}>
                <strong>{c.sector} - {c.district}</strong>
                <span className={`skill-tag ${c.urgency === 'High' ? 'urgent' : ''}`}>{c.urgency}</span>
            </div>
            <p>{c.description}</p>
        </div>
    ))}
</div>

```

```

        <p style={{ fontSize: '0.85rem', color: 'var(--text-m
        {c.emergingSkills && <p style={{ fontSize: '0.85rem',
        <p style={{ fontSize: '0.8rem', color: 'var(--text-mu
    </div>
    )})}
  </div>
</div>
)}

{ /*    VALIDATE REPORTS TAB    */}
{activeTab === 'validate' && (
    <ValidateReports />
)}
</div>
);
}

```

```

function ValidateReports() {
  const [reports, setReports] = useState([]);
  const [validatingId, setValidatingId] = useState(null);
  const [noteMap, setNoteMap] = useState({});
  const [statusMap, setStatusMap] = useState({});
  const token = localStorage.getItem('token');

  useEffect(() => {

```

```

axios.get(`${API}/api/reports`).then(r => {
  setReports(r.data);
  // Parse existing validation statuses from demandAnalysis
  const map = {};
  r.data.forEach(rep => {
    try {
      const d = JSON.parse(rep.demandAnalysis || '{}');
      if (d.validationStatus) map[rep.id] = d.validationStatus;
    } catch(e) {}
  });
  setStatusMap(map);
}).catch(console.error);
}, []);

const handleValidate = async (reportId, action) => {
  setValidatingId(reportId);
  try {
    const res = await axios.post(`${API}/api/reports/${reportId}/va
      { action, note: noteMap[reportId] || '' },
      { headers: { Authorization: `Bearer ${token}` } }
    );
    setStatusMap(prev => ({ ...prev, [reportId]: res.data.status }));
    alert(res.data.message);
  } catch (err) {
    alert(err.response?.data?.message || 'Failed to submit validation');
  }
}

```

```

    } finally {
        setValidatingId(null);
    }
};

return (
    <div className="glass-panel">
        <h2 style={{ marginBottom: '0.5rem' }}> Validate Gap Reports</h2>
        <p style={{ color: 'var(--text-muted)', marginBottom: '1.5rem',
            Review AI-generated gap reports and confirm whether the ident
        </p>
        {reports.length === 0 ? (
            <p style={{ color: 'var(--text-muted)' }}>No reports availabl
        ) : (
            reports.map(r => {
                const status = statusMap[r.id];
                return (
                    <div key={r.id} style={{ background: 'rgba(0,0,0,0.2)', p
                        <div style={{ display: 'flex', justifyContent: 'space-b
                            <div>
                                <h3 style={{ marginBottom: '0.2rem' }}>{r.college?.
                                {r.syllabus?.branch} (Sem {r.syllabus?.semester})</h3>
                                <span style={{ fontSize: '0.8rem', color: 'var(--te
                            </div>
                            {status ? (

```

```

      <span className={`skill-tag ${status === 'Employer
        {status === 'Employer Validated' ? '' : ''} {sta
      </span>
    ) : null}
  </div>

  <div style={{ marginBottom: '0.5rem' }}>
    <strong style={{ fontSize: '0.8rem', color: 'var(--ac
      {r.missingSkills.split(',').filter(s => s.trim()).map
    </div>

  <div style={{ marginBottom: '1rem' }}>
    <strong style={{ fontSize: '0.8rem', color: 'var(--ac
      {r.matchedSkills.split(',').filter(s => s.trim()).map
    </div>

  <input
    placeholder="Optional note (e.g. 'Confirmed -
we see this gap in every candidate')"
    value={noteMap[r.id] || ''}
    onChange={e => setNoteMap(prev => ({ ...prev, [r.id]:
      style={{ marginBottom: '0.75rem', fontSize: '0.85rem'
  />

  <div style={{ display: 'flex', gap: '0.5rem' }}>
    <button
      className="btn-sm btn-success"

```

```

        disabled={validatingId === r.id}
        onClick={() => handleValidate(r.id, 'validate')}
      >
        {validatingId === r.id ? '...' : ' Validate -
This is accurate'}
      </button>
    <button
      className="btn-sm btn-danger"
      disabled={validatingId === r.id}
      onClick={() => handleValidate(r.id, 'dispute')}
    >
      {validatingId === r.id ? '...' : ' Dispute -
This is wrong'}
    </button>
  </div>
</div>
);
})
)}
</div>
);
}

```

Source Code: StudentDashboard.jsx

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';
import API from '../api';

export default function StudentDashboard() {
  const [activeTab, setActiveTab] = useState('gaps');
  const [reports, setReports] = useState([]);
  const [trends, setTrends] = useState(null);
  const [jobs, setJobs] = useState([]);
  const navigate = useNavigate();

  // Personal assessment
  const [mySkills, setMySkills] = useState('');
  const [targetRole, setTargetRole] = useState('All');
  const [personalResult, setPersonalResult] = useState(null);
  const [analyzing, setAnalyzing] = useState(false);
  const [pathways, setPathways] = useState([]);

  useEffect(() => {
    const token = localStorage.getItem('token');
    if (!token) navigate('/login');
    fetchData();
  });
```

```
}, []);
```

```
const fetchData = async () => {  
  try {  
    const [r1, r2, r3, r4] = await Promise.all([  
      axios.get(`${API}/api/reports`),  
      axios.get(`${API}/api/skills/trends`).catch(() => ({ data: null })),  
      axios.get(`${API}/api/jobs`),  
      axios.get(`${API}/api/career-pathways`).catch(() => ({ data: null })),  
    ]);  
    setReports(r1.data);  
    setTrends(r2.data);  
    setJobs(r3.data);  
    setPathways(r4.data);  
  } catch (e) { console.error(e); }  
};
```

```
// Aggregate missing skills from all reports
```

```
const getTopMissingSkills = () => {  
  const freq = {};  
  reports.forEach(r => {  
    r.missingSkills.split(',').forEach(s => {  
      const t = s.trim().toLowerCase();  
      if (t) freq[t] = (freq[t] || 0) + 1;  
    });  
  });  
};
```

```

});

return Object.entries(freq).sort(([a],[b]) => b - a).slice(0, 1);
};

// Personal skill assessment
const handlePersonalAssessment = async (e) => {
  e.preventDefault();
  setAnalyzing(true);
  try {
    const filteredJobs = targetRole === 'All' ? jobs : jobs.filter(
    if (filteredJobs.length === 0) {
      alert('No active jobs found for this role.');
```

```

      setAnalyzing(false);
      return;
    }

    const allJobReqs = filteredJobs.map(j => `${j.title}: ${j.requirements}`);
    const res = await axios.post(`http://localhost:5001/analyze`, {
      syllabusText: mySkills,
      jobText: allJobReqs
    });

    setPersonalResult(res.data);
  } catch (err) {
    alert('ML engine not available');
  } finally {
    setAnalyzing(false);
  }
};

```

```

    }
};

const topMissing = getTopMissingSkills();

return (
  <div className="dashboard-container animate-fade-in">
    <div className="dashboard-header">
      <h1> Student Upskilling Portal</h1>
      <button className="btn-outline" onClick={() => { localStorage
    </div>

    <div className="tabs">
      <button className={`tab ${activeTab === 'gaps' ? 'active' : ''}
      <button className={`tab ${activeTab === 'assess' ? 'active' : ''}
      <button className={`tab ${activeTab === 'careers' ? 'active' : ''}
      <button className={`tab ${activeTab === 'trends' ? 'active' : ''}
    </div>

    {/*    MARKET SKILL GAPS TAB    */}
    {activeTab === 'gaps' && (
      <div className="dashboard-grid grid-2">
        <div className="glass-panel">
          <h2 style={{ marginBottom: '1rem', color: 'var(--accent-d
          <p style={{ color: 'var(--text-muted)', marginBottom: '1.

```

```

    These are the most frequently demanded skills by employ
</p>
{topMissing.length === 0 ? (
  <p style={{ color: 'var(--text-muted)' }}>No gap data a
) : (
  topMissing.map(([skill, count]) => (
    <div key={skill} style={{ display: 'flex', justifyCon
      <span style={{ fontWeight: 600, textTransform: 'cap
      <div style={{ display: 'flex', alignItems: 'center'
        <div style={{ background: 'var(--accent-danger)',
          <span style={{ fontSize: '0.8rem', color: 'var(--
        </div>
      </div>
    ))
  })
</div>

<div className="glass-panel">
  <h2 style={{ marginBottom: '1rem' }}> AI Recommended Cou
  <p style={{ color: 'var(--text-muted)', marginBottom: '1r
  {reports.length > 0 && (() => {
    let recs = [];
    try { recs = JSON.parse(reports[0].recommendations || '
    return recs.slice(0, 8).map((rec, i) => (
      <div key={i} style={{ background: 'rgba(0,0,0,0.3)',

```



```

<select
  value={targetRole}
  onChange={e => setTargetRole(e.target.value)}
  style={{ marginBottom: '1rem', width: '100%', padding: '5px' }}
>
  <option value="All">All Roles (Compare against entire database)
  { [...new Set(jobs.map(j => j.title))].map(role => (
    <option key={role} value={role}>{role}</option>
  ))}
</select>

<textarea
  placeholder="List your skills (e.g. Python, Java, SQL, etc.)"
  rows="6" value={mySkills} onChange={e => setMySkills(e.target.value)}
/>

<button type="submit" style={{ width: '100%' }} disabled={
  analyzing ? 'Analyzing...' : 'Run AI Assessment'
}>
</button>
</form>
</div>

<div className="glass-panel">
  <h2 style={{ marginBottom: '1rem' }}>Your Results</h2>
  {!personalResult ? (
    <p style={{ color: 'var(--text-muted)' }}>Enter your skills</p>
  ) : (

```

<>

```
<div className="stat-card" style={{ marginBottom: '1.5em' }}>
  <div className="stat-value" style={{ color: persona
    {personalResult.alignment_score}%
  }}>
  </div>
  <div className="stat-label">Industry Readiness Score</div>
  <div className="score-bar-bg" style={{ marginTop: '10px' }}>
    <div className="score-bar-fill" style={{
      width: `${personalResult.alignment_score}%`,
      background: personalResult.alignment_score >= 60 ? 'var(--accent-success)' : 'var(--accent-danger)',
    }}></div>
  </div>
</div>
```

```
<h3 style={{ color: 'var(--accent-danger)', marginBottom: '10px' }}>Missing Skills</h3>
<div style={{ display: 'flex', flexWrap: 'wrap', gap: '10px' }}>
  {(personalResult.missing_skills || []).map(s => <span>{s}</span>)}
</div>
```

```
<h3 style={{ color: 'var(--accent-success)', marginBottom: '10px' }}>Matched Skills</h3>
<div style={{ display: 'flex', flexWrap: 'wrap', gap: '10px' }}>
  {(personalResult.matched_skills || []).map(s => <span>{s}</span>)}
</div>
```

```
{personalResult.recommendations && personalResult.recommendations.length > 0 ? (
  <div>
```

```

<div style={{ marginTop: '1.5rem' }}>
  <h3 style={{ marginBottom: '0.5rem', fontSize: '0.8rem' }}>
    {personalResult.recommendations.slice(0, 5).map((rec) => (
      <div key={i} style={{ background: 'rgba(0,0,0,0.05)' }}>
        <strong>{rec.course}</strong> - {rec.provider}</div>
      </div>
    ))}
  </div>
)}
</>
)}
</div>
</div>
)}

```

```

{/* CAREER PATHWAYS TAB */}
{activeTab === 'careers' && (
  <div>
    <h2 style={{ marginBottom: '1.5rem' }}> Career Pathways (Landing Page)
    <p style={{ color: 'var(--text-muted)', marginBottom: '2rem' }}>
      <div className="dashboard-grid grid-3">
        {pathways.length === 0 ? <p style={{ color: 'var(--text-muted)' }}>
          No career pathways found.
        </p> :
        <div key={path.title} className="glass-panel">
          <div style={{ fontSize: '2rem', marginBottom: '0.5rem' }}>
            <h3 style={{ marginBottom: '0.5rem' }}>{path.title}</h3>

```

```

        <div style={{ display: 'flex', justifyContent: 'space-between' }}
            <span style={{ fontSize: '0.8rem', color: 'var(--text-muted)' }}>Trend data load</span>
            <span className="skill-tag rising">{path.demand}</span>
        </div>
        <div style={{ display: 'flex', flexWrap: 'wrap', gap: '10px' }}>
            {path.skills.map(s => <span key={s} className="skill-tag">{s}</span>)}
        </div>
    </div>
    )})
</div>
</div>
)}

```

```

{ /* SKILL TRENDS TAB */ }
{ activeTab === 'trends' && (
    <div>
        { !trends ? (
            <div className="glass-panel" style={{ textAlign: 'center' }}>
                <p style={{ color: 'var(--text-muted)' }}>Trend data load</p>
            </div>
        ) : (
            <div className="dashboard-grid grid-3" style={{ margin: '10px 0' }}>
                <div className="stat-card">
                    <div className="stat-value" style={{ color: 'var(--text-muted)' }}>

```

```

    <div className="stat-label">Rising Skills</div>
  </div>
  <div className="stat-card">
    <div className="stat-value" style={{ color: 'var(--color-rose-500)' }}>
      <div className="stat-label">Stable Skills</div>
    </div>
    <div className="stat-card">
      <div className="stat-value" style={{ color: 'var(--color-rose-500)' }}>
        <div className="stat-label">Declining Skills</div>
      </div>
    </div>
  </div>

<div className="dashboard-grid grid-2">
  <div className="glass-panel">
    <h2 style={{ marginBottom: '1rem', color: 'var(--color-rose-500)' }}>Trends
    <div style={{ maxHeight: '400px', overflowY: 'auto' }}>
      {Object.entries(trends.rising_by_category || {}).map(
        <div key={cat} style={{ marginBottom: '1rem' }}>
          <h4 style={{ fontSize: '0.85rem', color: 'var(--color-rose-500)' }}>{cat}
          <div style={{ display: 'flex', flexWrap: 'wrap' }}>
            {skills.map(s => <span key={s} className="stat-label">{s}</span>)}
          </div>
        </div>
      )}
    </div>
  </div>
</div>

```

```

    </div>
    <div className="glass-panel">
      <h2 style={{ marginBottom: '1rem', color: 'var(--accent)' }}>
      <p style={{ color: 'var(--text-muted)', marginBottom: '1rem' }}>
      <div style={{ display: 'flex', flexWrap: 'wrap', gap: '10px' }}>
        {(trends.declining || []).map(s => <span key={s}>
      </div>
    </div>
  </div>
</div>
</>
  )}
</div>
  )}
</div>
);
}

```

Source Code: Home.jsx

```

import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import axios from 'axios';
import API from '../api';

export default function Home() {

```

```

const [stats, setStats] = useState(null);

useEffect(() => {
  axios.get(`${API}/api/analytics/dashboard`).then(r => setStats(r.
}, []));

return (
  <div className="animate-fade-in">
    <nav className="navbar">
      <div className="navbar-brand">SkillBridge AI</div>
      <div style={{ display: 'flex', gap: '0.75rem' }}>
        <Link to="/login"><button className="btn-outline">Sign In</
        <Link to="/admin"><button className="btn-sm" style={{ backg
      </div>
    </nav>

    {/* Hero Section */}
    <main style={{ padding: '4rem 2rem', textAlign: 'center', maxWi
      <h1 style={{ fontSize: '3.2rem', fontWeight: 800, lineHeight:
        Bridging the Gap Between<br />Academia & Industry
      </h1>
      <p style={{ color: 'var(--text-muted)', fontSize: '1.15rem',
        A labour-market intelligence platform that combines job-pos
      </p>

```

```

<div style={{ display: 'flex', gap: '1rem', justifyContent: 'space-around' }}>
  <Link to="/login?role=STUDENT">
    <button style={{ padding: '0.8rem 2rem' }}>I'm a Student</button>
  </Link>
  <Link to="/login?role=COLLEGE">
    <button className="btn-success" style={{ padding: '0.8rem 2rem' }}>I'm a College Student</button>
  </Link>
  <Link to="/login?role=COMPANY">
    <button className="btn-warning" style={{ padding: '0.8rem 2rem' }}>I'm a Company Employee</button>
  </Link>
</div>

```

```

{/* Live Stats */}

```

```

{stats && (
  <div className="dashboard-grid grid-4" style={{ marginBottom: 20 }}>
    <div className="stat-card animate-count">
      <div className="stat-value">{stats.totalJobs || 0}</div>
      <div className="stat-label">Job Postings Analyzed</div>
    </div>
    <div className="stat-card animate-count">
      <div className="stat-value">{stats.totalReports || 0}</div>
      <div className="stat-label">Gap Reports Generated</div>
    </div>
    <div className="stat-card animate-count">
      <div className="stat-value">{stats.avgAlignment || 0}%</div>
      <div className="stat-label">Average Alignment</div>
    </div>
  </div>
)
}

```

```

        <div className="stat-label">Avg Curriculum Alignment</div>
    </div>

    <div className="stat-card animate-count">
        <div className="stat-value">{stats.totalUsers || 0}</div>
        <div className="stat-label">Registered Users</div>
    </div>
</div>
)}}

```

```

{/* Feature Cards */}

<h2 style={{ marginBottom: '2rem', color: 'var(--text-muted)' }}>
    <div className="dashboard-grid grid-3">
        <div className="glass-panel" style={{ textAlign: 'left' }}>
            <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>
                <h3 style={{ marginBottom: '0.5rem' }}>Labour Market Intelligence
                <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
                    Analyzes job postings to identify demand by role, skill
                </p>
            </div>
        </div>
        <div className="glass-panel" style={{ textAlign: 'left' }}>
            <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>
                <h3 style={{ marginBottom: '0.5rem' }}>Curriculum Gap Analysis
                <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
                    Maps skill gaps to courses, recommends curriculum updates
                </p>
            </div>
        </div>
    </div>

```

```

</div>
<div className="glass-panel" style={{ textAlign: 'left' }}>
  <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>
    <h3 style={{ marginBottom: '0.5rem' }}>District Training
    <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}
      Generates district-level training plans with trainer re
    </p>
  </div>
</div>
<div className="glass-panel" style={{ textAlign: 'left' }}>
  <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>
    <h3 style={{ marginBottom: '0.5rem' }}>Employer Surveys</h3>
    <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}
      Collects employer satisfaction, candidate quality rating
    </p>
  </div>
</div>
<div className="glass-panel" style={{ textAlign: 'left' }}>
  <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>
    <h3 style={{ marginBottom: '0.5rem' }}>Placement Tracking
    <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}
      Tracks placement outcomes by role, location, and package
    </p>
  </div>
</div>
<div className="glass-panel" style={{ textAlign: 'left' }}>
  <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>
    <h3 style={{ marginBottom: '0.5rem' }}>Career Pathways</h3>

```

```

        <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
            Helps students identify high-demand skills, get personal
        </p>
    </div>
</div>
</main>
</div>
);
}

```

Source Code: AdminDashboard.jsx

```

import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';
import API from '../api';

export default function AdminDashboard() {
    const [stats, setStats] = useState(null);
    const [surveyStats, setSurveyStats] = useState(null);
    const [placementStats, setPlacementStats] = useState(null);
    const [trends, setTrends] = useState(null);
    const [seeding, setSeeding] = useState(false);
    const [seedMsg, setSeedMsg] = useState('');
    const navigate = useNavigate();

```

```

useEffect(() => { fetchAll(); }, []);

const fetchAll = async () => {
  try {
    const [s1, s2, s3, s4] = await Promise.all([
      axios.get(`${API}/api/analytics/dashboard`).catch(() => ({ data: null })),
      axios.get(`${API}/api/surveys/stats`).catch(() => ({ data: null })),
      axios.get(`${API}/api/placements/stats`).catch(() => ({ data: null })),
      axios.get(`${API}/api/skills/trends`).catch(() => ({ data: null })),
    ]);
    setStats(s1.data);
    setSurveyStats(s2.data);
    setPlacementStats(s3.data);
    setTrends(s4.data);
  } catch (e) { console.error(e); }
};

const handleSeed = async () => {
  setSeeding(true);
  try {
    await axios.post(`${API}/api/seed`);
    setSeedMsg('Demo data loaded! Refresh to see stats.');
```

fetchAll();

```

  } catch (e) { setSeedMsg('Seed failed'); }
};

```

```

    finally { setSeeding(false); }
};

return (
  <div className="dashboard-container animate-fade-in">
    <div className="dashboard-header">
      <h1> Neural Analytics Dashboard</h1>
      <div style={{ display: 'flex', gap: '0.75rem' }}>
        <button className="btn-warning btn-sm" onClick={handleSeed}
          {seeding ? 'Seeding...' : ' Boot Neural Data'}
        </button>
        <button className="btn-outline" onClick={() => navigate('/')}>
      </div>
    </div>

    {seedMsg && <div style={{ background: 'rgba(16,185,129,0.15)',

    {/*    OVERVIEW STATS    */}
    {stats && (
      <>
        <div className="dashboard-grid grid-4" style={{ marginBotto
          <div className="stat-card animate-count">
            <div className="stat-value">{stats.totalUsers}</div>
            <div className="stat-label">Total Users</div>
          </div>

```

```

<div className="stat-card animate-count">
  <div className="stat-value">{stats.totalJobs}</div>
  <div className="stat-label">Job Postings</div>
</div>

<div className="stat-card animate-count">
  <div className="stat-value">{stats.totalReports}</div>
  <div className="stat-label">Gap Reports</div>
</div>

<div className="stat-card animate-count">
  <div className="stat-value">{stats.avgAlignment}%</div>
  <div className="stat-label">Avg Alignment Score</div>
</div>
</div>

<div className="dashboard-grid grid-3" style={{ marginBotto
  <div className="stat-card">
    <div className="stat-value" style={{ fontSize: '1.5rem'
    <div className="stat-label">Students</div>
  </div>
  <div className="stat-card">
    <div className="stat-value" style={{ fontSize: '1.5rem'
    <div className="stat-label">Colleges</div>
  </div>
  <div className="stat-card">
    <div className="stat-value" style={{ fontSize: '1.5rem'

```

```

        <div className="stat-label">Companies</div>
      </div>
    </div>
  </>
)}

<div className="dashboard-grid grid-2" style={{ marginBottom: '
  { /*    TOP MISSING SKILLS    */ }
  <div className="glass-panel">
    <h2 style={{ marginBottom: '1rem', color: 'var(--accent-dan
    {stats?.topMissingSkills?.length > 0 ? (
      stats.topMissingSkills.map((item, i) => (
        <div key={item.skill} style={{ display: 'flex', justify
          <div style={{ display: 'flex', alignItems: 'center',
            <span style={{ color: 'var(--text-muted)', fontSize
            <span style={{ fontWeight: 600, textTransform: 'cap
          </div>
        <div style={{ display: 'flex', alignItems: 'center',
          <div style={{ background: 'var(--accent-danger)', h
          <span style={{ fontSize: '0.8rem', color: 'var(--te
        </div>
      </div>
    ))
  ) : <p style={{ color: 'var(--text-muted)' }}>No gap data y
</div>

```

```

{ /*    EMPLOYER SURVEY STATS    */}
<div className="glass-panel">
  <h2 style={{ marginBottom: '1rem', color: 'var(--accent-war
  {surveyStats ? (
    <>
      <div className="dashboard-grid grid-2" style={{ marginB
        <div className="stat-card">
          <div className="stat-value" style={{ fontSize: '1.8
            {surveyStats.avgSatisfaction}/5
          </div>
          <div className="stat-label">Avg Satisfaction</div>
        </div>
      <div className="stat-card">
        <div className="stat-value" style={{ fontSize: '1.8
        <div className="stat-label">Total Surveys</div>
      </div>
    </div>

    {surveyStats.topNeededSkills?.length > 0 && (
      <>
        <h3 style={{ fontSize: '0.9rem', marginBottom: '0.7
        <div style={{ display: 'flex', flexWrap: 'wrap', ga
          {surveyStats.topNeededSkills.map(item => (
            <span key={item.skill} className="skill-tag mis

```

```

        {item.skill} ({item.count})
      </span>
    )))
  </div>
</>
})

```

```

{surveyStats.qualityDistribution && Object.keys(surveyStats.qualityDistribution)}
  <div style={{ marginTop: '1.5rem' }}>
    <h3 style={{ fontSize: '0.9rem', marginBottom: '0.7rem' }}>Quality Distribution
    {Object.entries(surveyStats.qualityDistribution).map((entry) => {
      const colors = { 'Poor': '#ef4444', 'Below Average': '#ffc107', 'Average': '#6c757d', 'Above Average': '#28a745', 'Excellent': '#6f42c1' };
      return (
        <div key={entry[0]} style={{ display: 'flex', align-items: 'center', gap: '10px' }}>
          <span style={{ fontSize: '0.85rem', width: '150px' }}>{entry[0]}</span>
          <div style={{ flex: 1, height: '8px', background: `linear-gradient(to right, ${entry[1]} 0%, ${entry[1]} 100%)` }}>
            <div style={{ height: '100%', width: `${(entry[1] / 100).toFixed(1)}%` }}>
              </div>
            </div>
          <span style={{ fontSize: '0.8rem', color: 'var(--color)', font-weight: 'bold' }}>{entry[1]}</span>
        </div>
      );
    })}
  </div>
})
</>

```

```

    ) : <p style={{ color: 'var(--text-muted)' }}>No survey data
  </div>
</div>

<div className="dashboard-grid grid-2">
  {/*    PLACEMENT STATS    */}
  <div className="glass-panel">
    <h2 style={{ marginBottom: '1rem', color: 'var(--accent-success)' }}>
      {placementStats ? (
        <div className="dashboard-grid grid-3">
          <div className="stat-card">
            <div className="stat-value" style={{ fontSize: '1.5rem' }}>
              <div className="stat-label">Students Placed</div>
            </div>
          <div className="stat-card">
            <div className="stat-value" style={{ fontSize: '1.5rem' }}>
              <div className="stat-label">Total Students</div>
            </div>
          <div className="stat-card">
            <div className="stat-value" style={{ fontSize: '1.5rem' }}>
              <div className="stat-label">Placement Rate</div>
            </div>
          </div>
        </div>
      ) : <p style={{ color: 'var(--text-muted)' }}>No placement
    </div>
  }

```

```

{ /*    SKILL TRENDS    */}
<div className="glass-panel">
  <h2 style={{ marginBottom: '1rem', color: 'var(--secondary)' }}>
    {trends ? (
      <>
        <div className="dashboard-grid grid-3" style={{ marginB
          <div style={{ textAlign: 'center' }}>
            <div style={{ fontSize: '1.5rem', fontWeight: 700,
              <div style={{ fontSize: '0.75rem', color: 'var(--te
            </div>
          <div style={{ textAlign: 'center' }}>
            <div style={{ fontSize: '1.5rem', fontWeight: 700,
              <div style={{ fontSize: '0.75rem', color: 'var(--te
            </div>
          <div style={{ textAlign: 'center' }}>
            <div style={{ fontSize: '1.5rem', fontWeight: 700,
              <div style={{ fontSize: '0.75rem', color: 'var(--te
            </div>
          </div>
        <h4 style={{ fontSize: '0.85rem', marginBottom: '0.5rem' }}>
          <div style={{ display: 'flex', flexWrap: 'wrap', gap: '10px' }}>
            {(trends.top_emerging || []).map(s => <span key={s} c
          </div>
        </div>
      </>
    )
  </h2>
</div>

```

```

    ) : <p style={{ color: 'var(--text-muted)' }}>Trends loading...
  </div>
</div>

```

```

{ /*    SECTOR GROWTH + PROFICIENCY DEMAND    */ }

<div className="dashboard-grid grid-2" style={{ marginTop: '2rem' }}>
  <div className="glass-panel">
    <h2 style={{ marginBottom: '1rem', color: 'var(--primary)' }}>
      Sector Growth + Proficiency Demand
    <p style={{ color: 'var(--text-muted)', fontSize: '0.85rem' }}>
      {stats?.sectorGrowth?.length > 0 ? (
        stats.sectorGrowth.map((item, i) => (
          <div key={item.sector} style={{ display: 'flex', alignI
            <span style={{ fontSize: '0.85rem', width: '110px', f
            <div style={{ flex: 1, height: '10px', background: 'r
              <div style={{ height: '100%', width: `${(item.count
            </div>
            <span style={{ fontSize: '0.8rem', color: 'var(--text
          </div>
        ))
      ) : <p style={{ color: 'var(--text-muted)' }}>No job data y
    </div>
  </div>

```

```

<div className="glass-panel">
  <h2 style={{ marginBottom: '1rem', color: 'var(--accent-war
  <p style={{ color: 'var(--text-muted)', fontSize: '0.85rem' }}>
    No job data yet
  </p>
</div>

```

```

{stats?.proficiencyDemand?.length > 0 ? (
  stats.proficiencyDemand.map((item) => {
    const colors = { Beginner: '#10b981', Intermediate: '#6
    const total = stats.proficiencyDemand.reduce((s, i) =>
    return (
      <div key={item.level} style={{ marginBottom: '1rem' }}
        <div style={{ display: 'flex', justifyContent: 'spa
          <span style={{ fontWeight: 600 }}>{item.level}</s
          <span style={{ fontSize: '0.85rem', color: 'var(--
        </div>
      <div style={{ height: '12px', background: 'rgba(255
        <div style={{ height: '100%', width: `${(item.cou
      </div>
    </div>
  );
})
) : <p style={{ color: 'var(--text-muted)' }}>No proficienc

{/* Quality indicators */}
{((stats?.validatedCount > 0 || stats?.revisedCount > 0) &&
  <div style={{ marginTop: '1.5rem', paddingTop: '1rem', bo
    <h4 style={{ fontSize: '0.85rem', marginBottom: '0.75re
    <div style={{ display: 'flex', gap: '1rem' }}>
      <div className="stat-card" style={{ flex: 1, padding:
        <div style={{ fontSize: '1.5rem', fontWeight: 700,

```

```

        <div style={{ fontSize: '0.75rem', color: 'var(--te
</div>
<div className="stat-card" style={{ flex: 1, padding:
    <div style={{ fontSize: '1.5rem', fontWeight: 700,
        <div style={{ fontSize: '0.75rem', color: 'var(--te
    </div>
</div>
</div>
    )}
</div>
</div>
</div>
);
}

```

Source Code: index.css

```

@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;

:root {
  --bg-primary: #05050f;
  --bg-secondary: #0a0b1e;
  --bg-card: rgba(10, 11, 30, 0.7);
  --bg-glass: rgba(139, 92, 246, 0.03);

```

```

--primary: #a855f7; /* AI Purple */
--primary-glow: rgba(168, 85, 247, 0.4);
--secondary: #00f0ff; /* Neon Cyan */
--accent-success: #22c55e;
--accent-warning: #eab308;
--accent-danger: #ef4444;
--text-main: #f8fafc;
--text-muted: #94a3b8;
--border: rgba(168, 85, 247, 0.2);
--gradient-1: linear-gradient(135deg, #a855f7, #ec4899);
--gradient-2: linear-gradient(135deg, #00f0ff, #3b82f6);
--gradient-3: linear-gradient(135deg, #eab308, #ef4444);
}

```

```

* { margin: 0; padding: 0; box-sizing: border-box; }

```

```

body {
  font-family: 'Inter', -apple-system, sans-serif;
  background: var(--bg-primary);
  color: var(--text-main);
  min-height: 100vh;
  -webkit-font-smoothing: antialiased;
}

```

```

/*    GLASS PANEL    */

```

```

.glass-panel {
  background: var(--bg-glass);
  backdrop-filter: blur(20px);
  border: 1px solid var(--border);
  border-radius: 1rem;
  padding: 1.5rem;
  transition: all 0.3s ease;
}

.glass-panel:hover {
  border-color: rgba(168, 85, 247, 0.4);
  box-shadow: 0 0 40px rgba(168, 85, 247, 0.1), inset 0 0 20px rgba(0
}

/*      NAVBAR      */

.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  border-bottom: 1px solid var(--border);
  background: rgba(10, 14, 26, 0.9);
  backdrop-filter: blur(10px);
  position: sticky;
  top: 0;
  z-index: 100;
}

```

```

}

.navbar-brand {
  font-size: 1.5rem;
  font-weight: 800;
  background: var(--gradient-1);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  letter-spacing: -0.5px;
}

/*      BUTTONS      */
button, .btn {
  padding: 0.6rem 1.5rem;
  border: none;
  border-radius: 0.5rem;
  font-weight: 600;
  font-size: 0.9rem;
  cursor: pointer;
  transition: all 0.2s ease;
  font-family: 'Inter', sans-serif;
  background: var(--primary);
  color: white;
}

button:hover { transform: translateY(-1px); opacity: 0.9; }
button:disabled { opacity: 0.5; cursor: not-allowed; transform: none; }

```

```

.btn-outline {
  background: transparent;
  border: 1px solid var(--border);
  color: var(--text-main);
}

.btn-outline:hover { border-color: var(--primary); }

.btn-success { background: var(--accent-success); }
.btn-warning { background: var(--accent-warning); color: #000; }
.btn-danger { background: var(--accent-danger); }
.btn-sm { padding: 0.4rem 1rem; font-size: 0.8rem; }

/*      INPUTS      */
input, textarea, select {
  width: 100%;
  padding: 0.75rem 1rem;
  border: 1px solid var(--border);
  border-radius: 0.5rem;
  background: rgba(0, 0, 0, 0.3);
  color: var(--text-main);
  font-family: 'Inter', sans-serif;
  font-size: 0.9rem;
  margin-bottom: 0.75rem;
  transition: border-color 0.2s;
}

```

```

}

input:focus, textarea:focus, select:focus {
    outline: none;
    border-color: var(--primary);
    box-shadow: 0 0 0 3px var(--primary-glow);
}

select option { background: var(--bg-secondary); color: var(--text-ma

/*    DASHBOARD LAYOUT                                */

.dashboard-container {
    max-width: 1400px;
    margin: 0 auto;
    padding: 2rem;
}

.dashboard-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 2rem;
}

.dashboard-grid {
    display: grid;
    gap: 1.5rem;
}

.grid-2 { grid-template-columns: 1fr 1fr; }

```

```

.grid-3 { grid-template-columns: 1fr 1fr 1fr; }
.grid-4 { grid-template-columns: repeat(4, 1fr); }

/*      STAT CARDS      */
.stat-card {
    background: var(--bg-glass);
    border: 1px solid var(--border);
    border-radius: 1rem;
    padding: 1.25rem;
    text-align: center;
    transition: all 0.3s ease;
}
.stat-card:hover { transform: translateY(-3px); border-color: var(--p
.stat-value {
    font-size: 2rem;
    font-weight: 800;
    background: var(--gradient-1);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}
.stat-label { font-size: 0.8rem; color: var(--text-muted); margin-top

/*      SKILL TAGS      */
.skill-tag {
    display: inline-block;

```

```

padding: 0.3rem 0.75rem;
border-radius: 2rem;
font-size: 0.8rem;
font-weight: 500;
margin: 0.2rem;
}

.skill-tag.missing { background: rgba(239, 68, 68, 0.15); color: #f87
.skill-tag.matched { background: rgba(16, 185, 129, 0.15); color: #34
.skill-tag.obsolete { background: rgba(245, 158, 11, 0.15); color: #f
.skill-tag.rising { background: rgba(99, 102, 241, 0.15); color: #818

/*      SCORE BAR      */

.score-bar-bg {
    height: 10px;
    background: rgba(255,255,255,0.1);
    border-radius: 999px;
    overflow: hidden;
    margin-top: 0.5rem;
}

.score-bar-fill {
    height: 100%;
    border-radius: 999px;
    transition: width 1s ease;
}

```

```

/*      TABS                                          */

.tabs { display: flex; gap: 0; border-bottom: 1px solid var(--border) }
.tab {
  padding: 0.75rem 1.5rem;
  cursor: pointer;
  color: var(--text-muted);
  border-bottom: 2px solid transparent;
  font-weight: 500;
  transition: all 0.2s;
  background: none;
  border-radius: 0;
}
.tab:hover { color: var(--text-main); }
.tab.active {
  color: var(--primary);
  border-bottom-color: var(--primary);
}

/*      ANIMATIONS                                  */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}
.animate-fade-in { animation: fadeIn 0.5s ease-out; }

```

```

@keyframes countUp {
  from { opacity: 0; transform: scale(0.5); }
  to { opacity: 1; transform: scale(1); }
}

.animate-count { animation: countUp 0.8s ease-out; }

@keyframes pulse { 0%, 100% { opacity: 1; } 50% { opacity: 0.5; } }
.animate-pulse { animation: pulse 2s ease-in-out infinite; }

/*      RESPONSIVE      */
@media (max-width: 900px) {
  .grid-2, .grid-3, .grid-4 { grid-template-columns: 1fr; }
  .dashboard-container { padding: 1rem; }
}

/*      SCROLLBAR      */
::-webkit-scrollbar { width: 6px; }
::-webkit-scrollbar-track { background: transparent; }
::-webkit-scrollbar-thumb { background: rgba(99, 102, 241, 0.3); border: 1px solid transparent; }

/*      TABLE      */
.data-table { width: 100%; border-collapse: collapse; }
.data-table th {
  text-align: left; padding: 0.75rem; color: var(--text-muted);
  font-size: 0.8rem; text-transform: uppercase; letter-spacing: 0.05em;
}

```

```
border-bottom: 1px solid var(--border);
}

.data-table td {
padding: 0.75rem; border-bottom: 1px solid var(--border);
font-size: 0.9rem;
}

.data-table tr:hover td { background: rgba(99, 102, 241, 0.05); }
```